

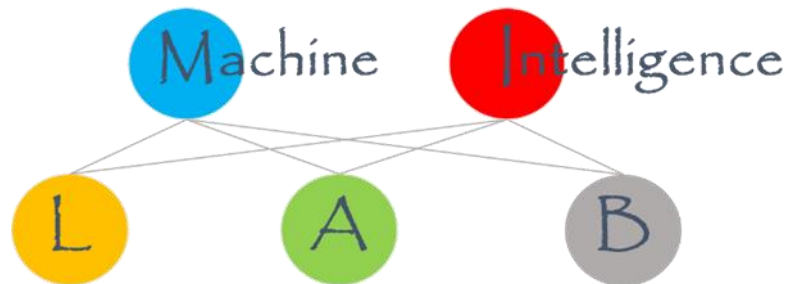


北京大學
PEKING UNIVERSITY

Pixel Computing – I

Mu Yadong

Wangxuan Institute of Computer Technology
Peking University



**Several slides are adapted from related courses or tutorials.
Internal use only. Please do not distribute the slides.**

Outline

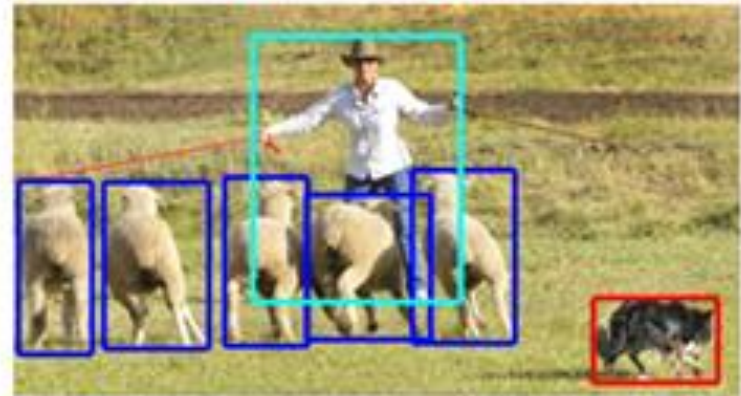
- Examples of Pixel Computing
- Unsupervised Image Segmentation
 - K-means, Mean-Shift, Normalized Cut
- Interactive Object Cutout
 - GraphCut, GrabCut, LazySnapping
- Image Matting
- Related Topics
 - Image Co-segmentation
 - Image Inpainting / Image Completion

Pixel Labeling

- Understanding an image at the pixel level



(a) classification



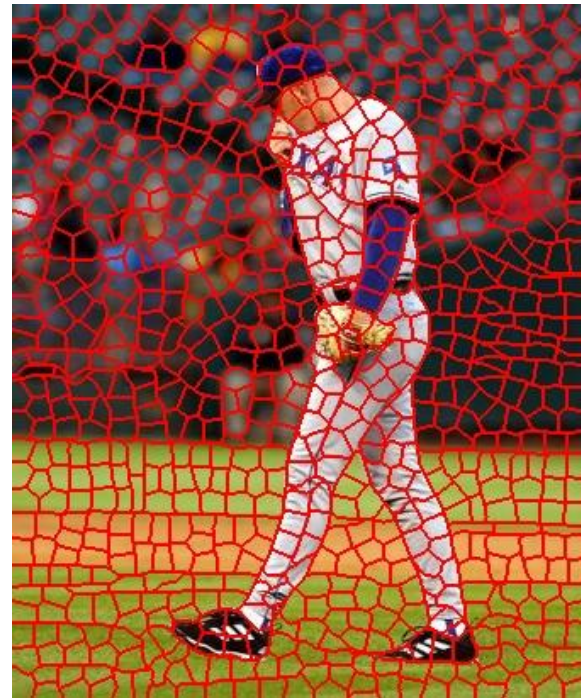
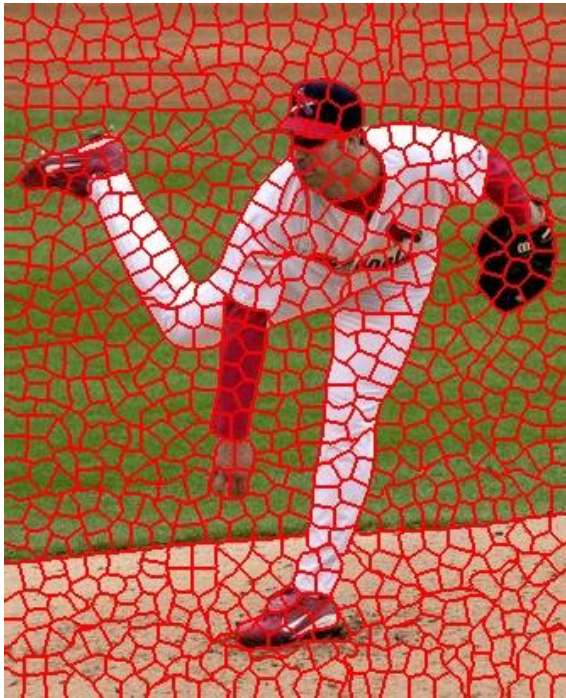
(b) detection



(c) segmentation

Unsupervised image segmentation

- Super-pixel: Group together similar-looking pixels for efficiency of further processing



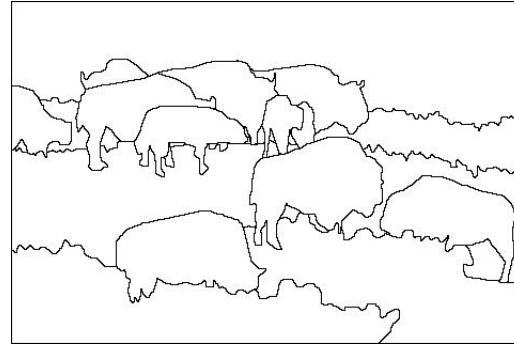
X. Ren and J. Malik. Learning a classification model for segmentation. ICCV 2003.

Unsupervised image segmentation

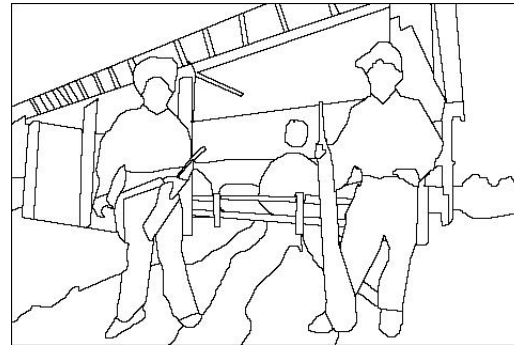
- Separate image into coherent “objects”



image



human segmentation



Berkeley segmentation database:

<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/>

Object Cutout and Image Matting

- Blue screen for movies



Object Cutout and Image Matting

- User interaction as the input (strokes or bound box)



Object Cutout and Image Matting

- Image Matting
- For semi-transparent or shining objects (for example, hair)



Object Cutout and Image Matting

- The “trimap”



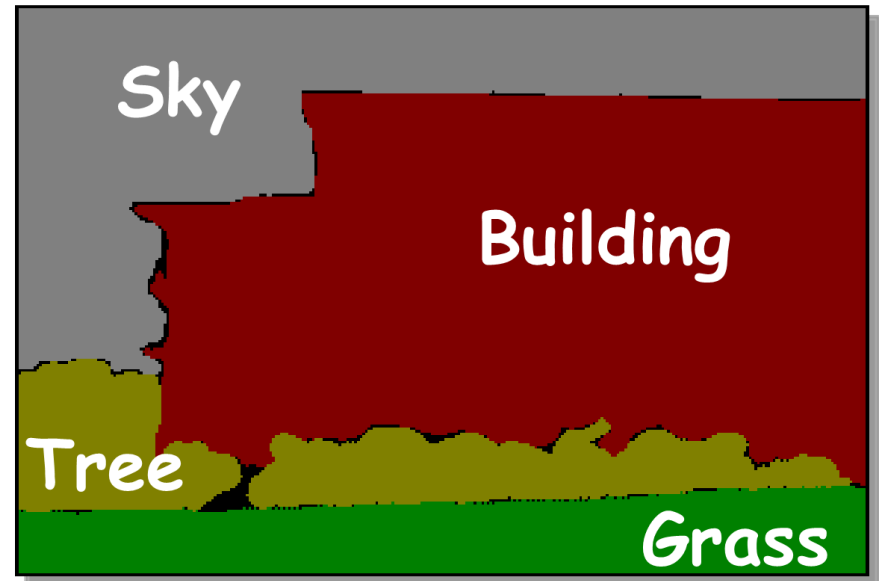
Image Co-Segmentation

- Segment common objects from a pair / group of images



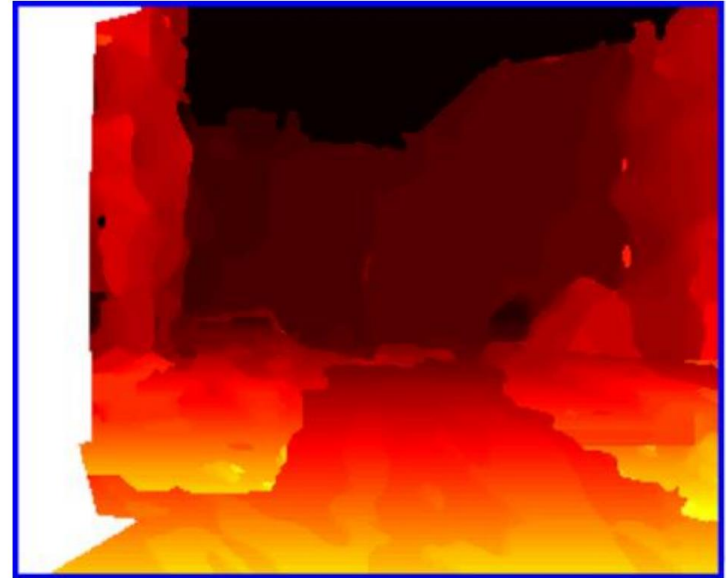
Image Parsing

- Associate each image pixel with a pre-specified semantic label
- Applications: autonomous driving, 3D reconstruction etc.



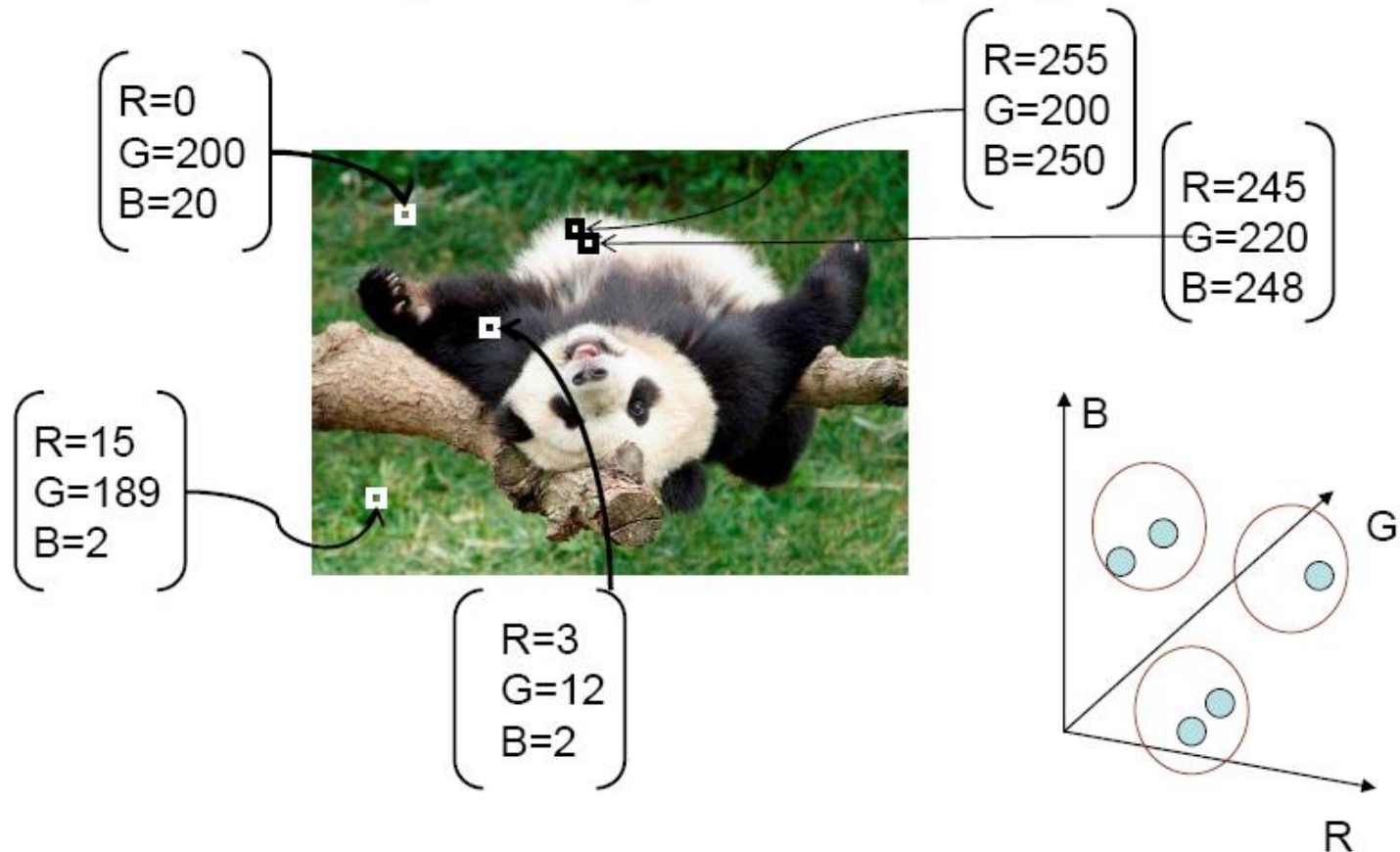
Other Pixel Labeling Problems

- Depth estimation from a single image



Segmentation as clustering

- Cluster similar (in terms of RGB color) pixels together



Segmentation as clustering

- K-means clustering based on intensity or color is essentially vector quantization of the image attributes
 - Clusters don't have to be spatially coherent

Image



Intensity-based clusters

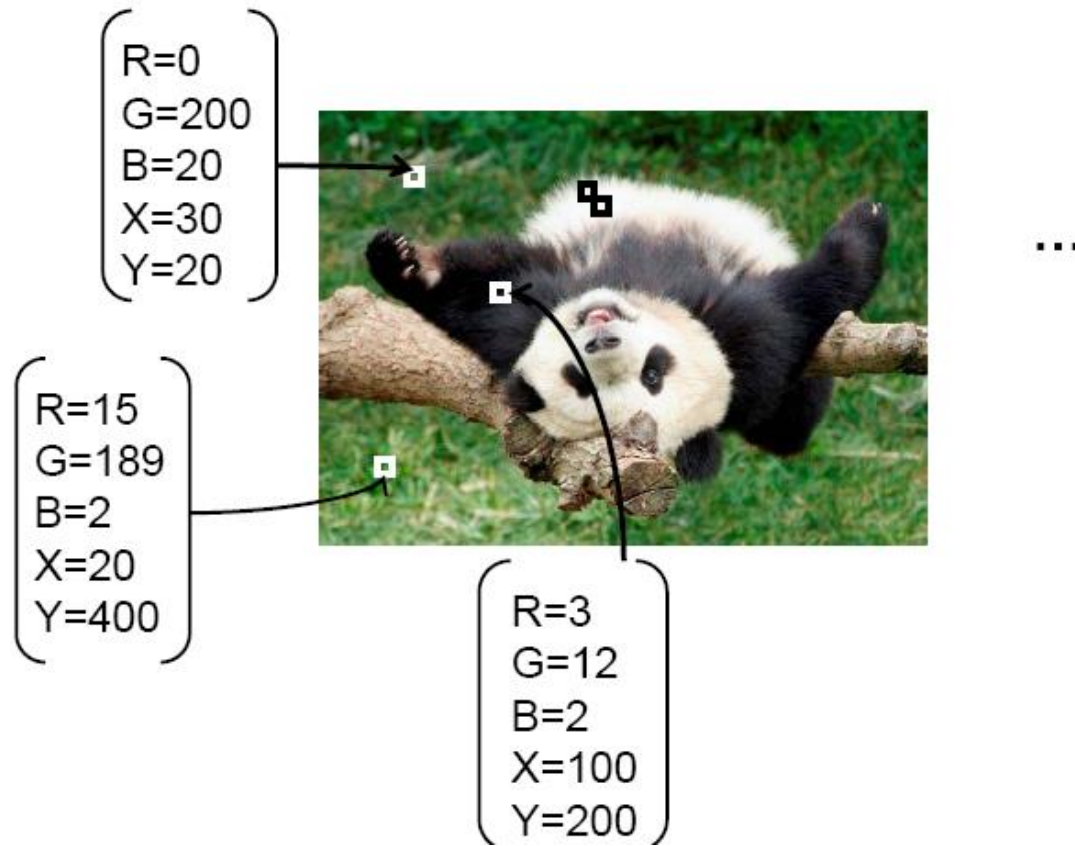


Color-based clusters



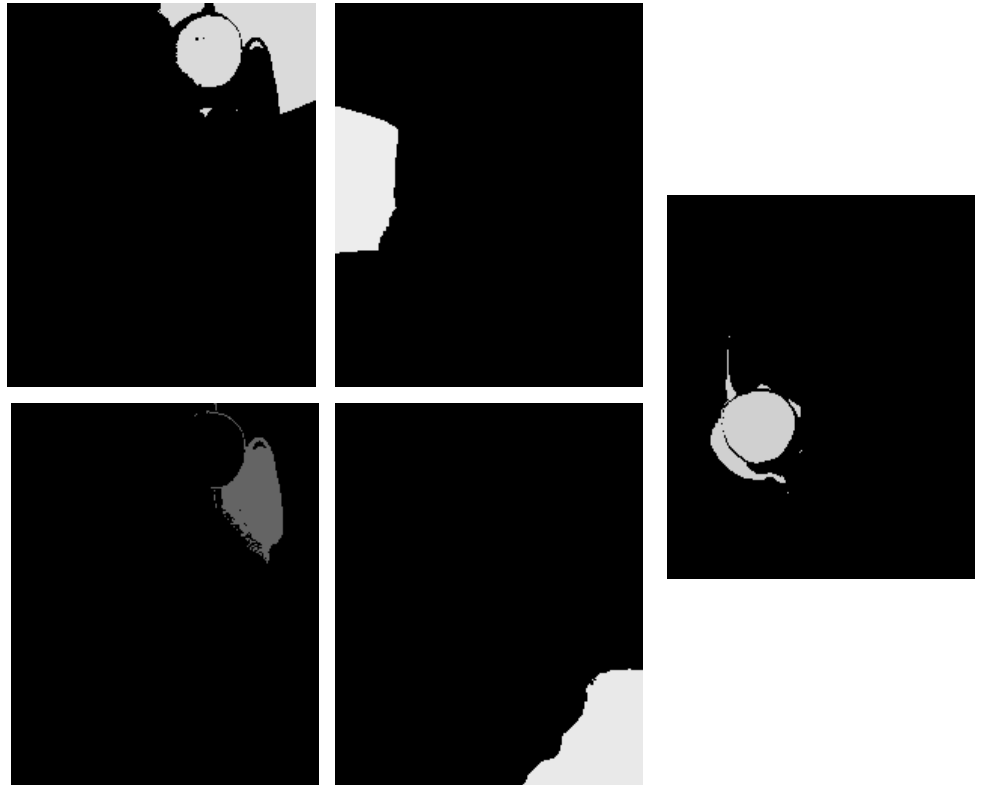
Segmentation as clustering

- Cluster similar pixels (features) together



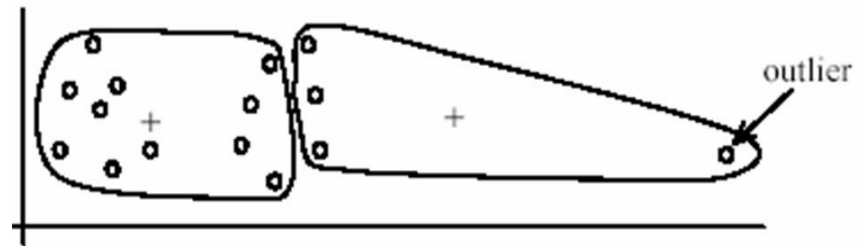
Segmentation as clustering

- Clustering based on (r, g, b, x, y) values enforces more spatial coherence



K-Means for segmentation

- Pros
 - Very simple method
 - Converges to a local minimum of the error function
- Cons
 - Memory-intensive
 - Need to pick K
 - Sensitive to initialization
 - Sensitive to outliers
 - Only finds “spherical” clusters



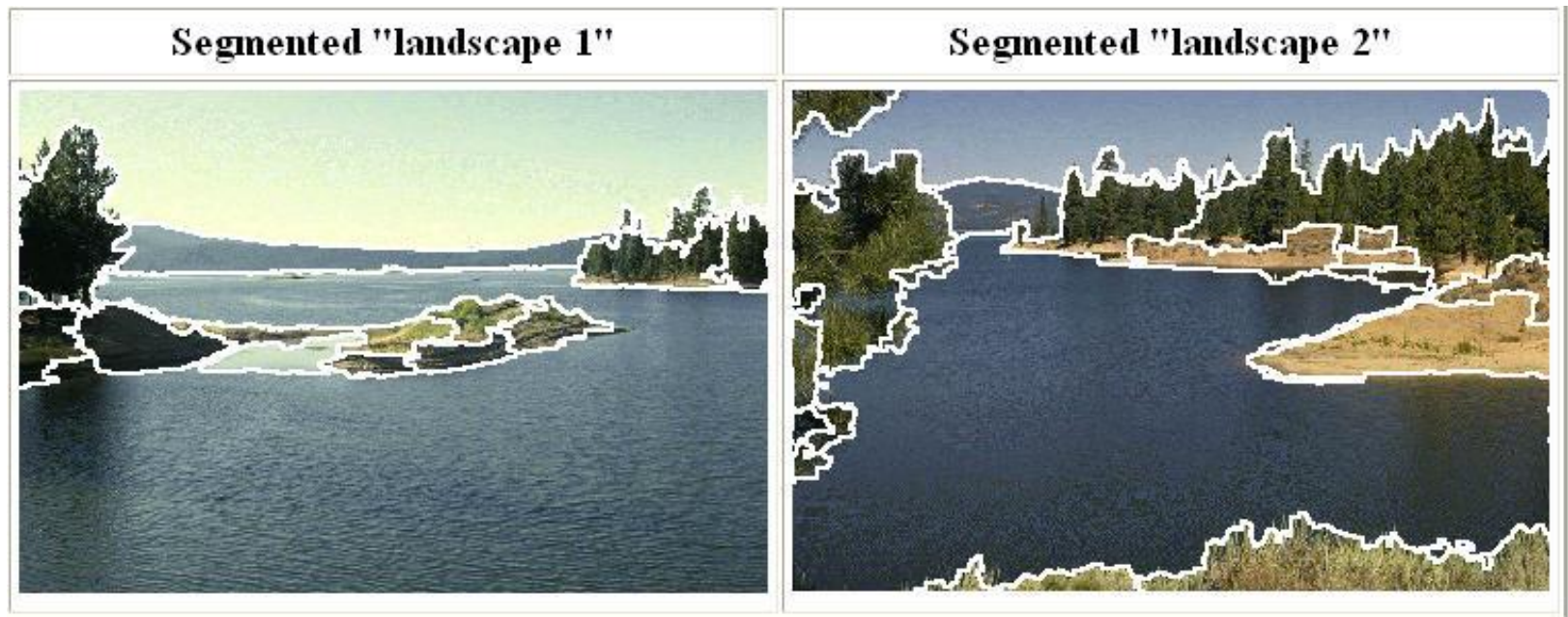
(A): Undesirable clusters



(B): Ideal clusters

Mean shift clustering and segmentation

- An advanced and versatile technique for clustering-based segmentation



<http://www.caip.rutgers.edu/~comanici/MSPAMI/msPamiResults.html>

D. Comaniciu and P. Meer, Mean Shift: A Robust Approach toward Feature Space Analysis, PAMI 2002.

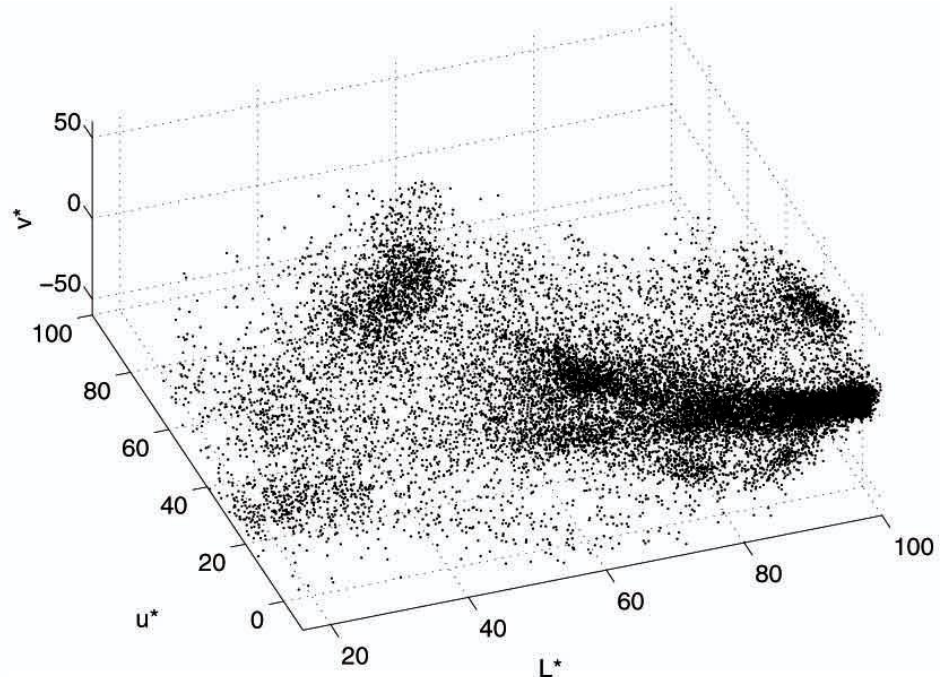
Mean shift algorithm

- The mean shift algorithm seeks *modes* or local maxima of density in the feature space

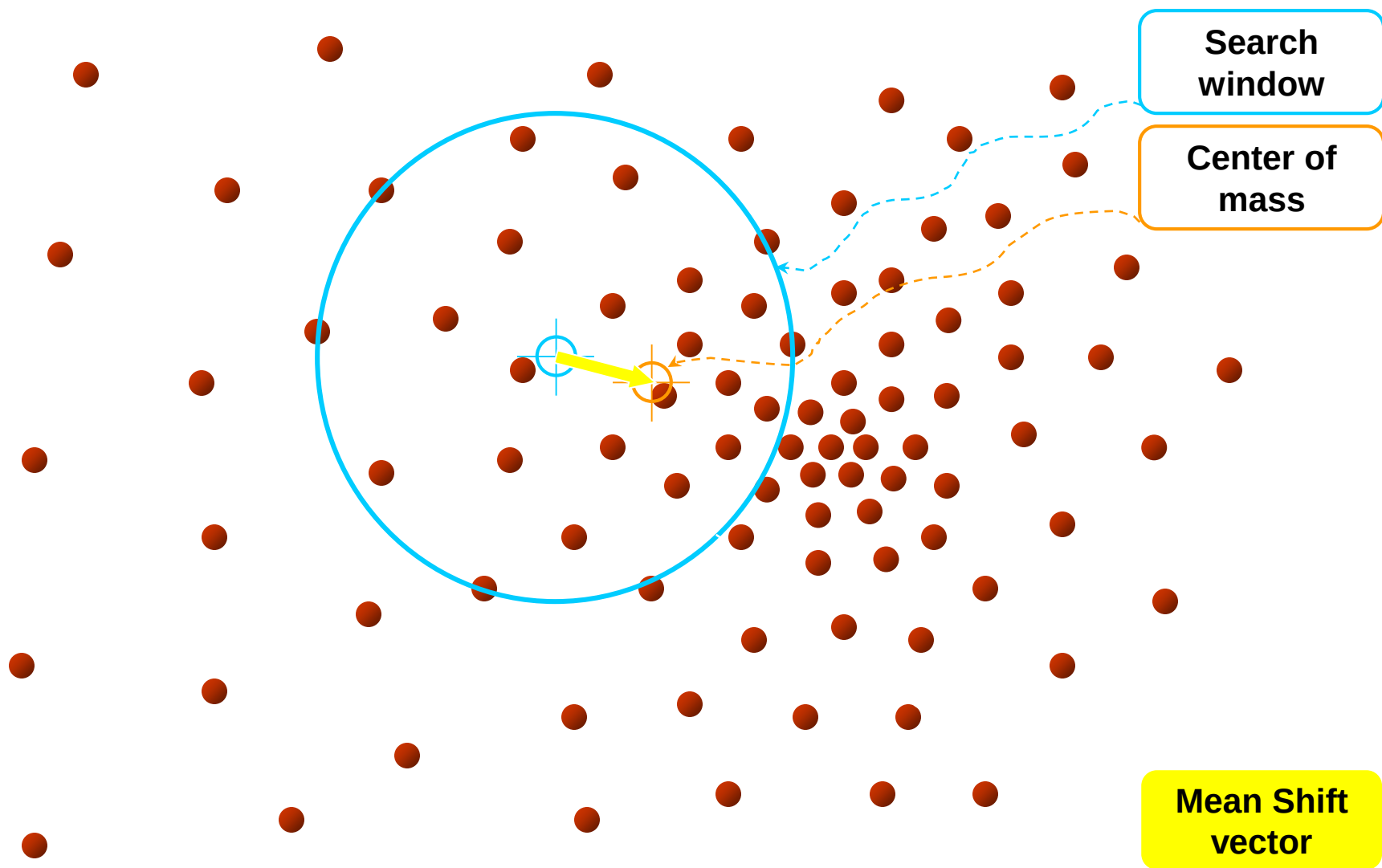
image



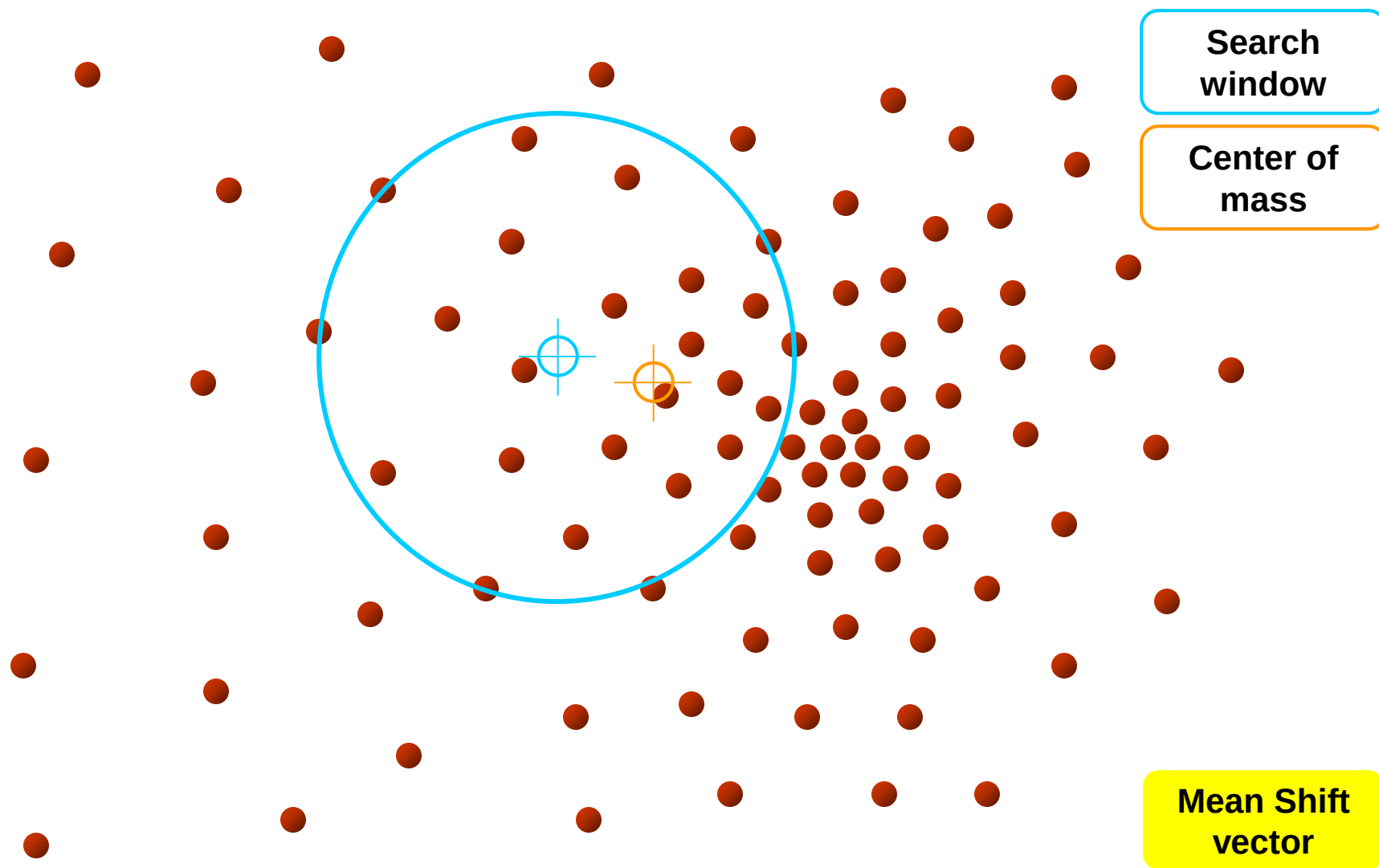
Feature space
($L^*u^*v^*$ color values)



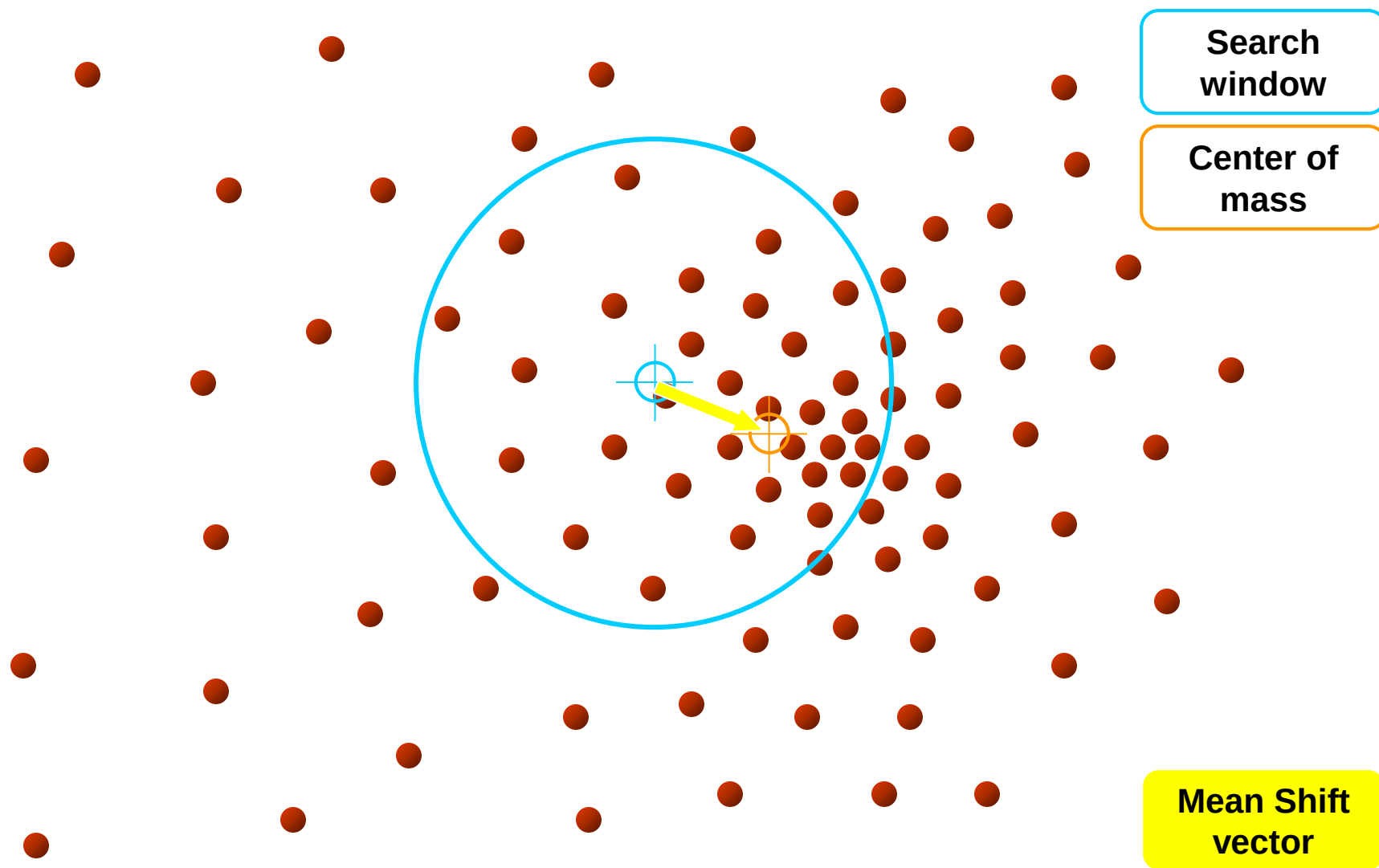
Mean shift



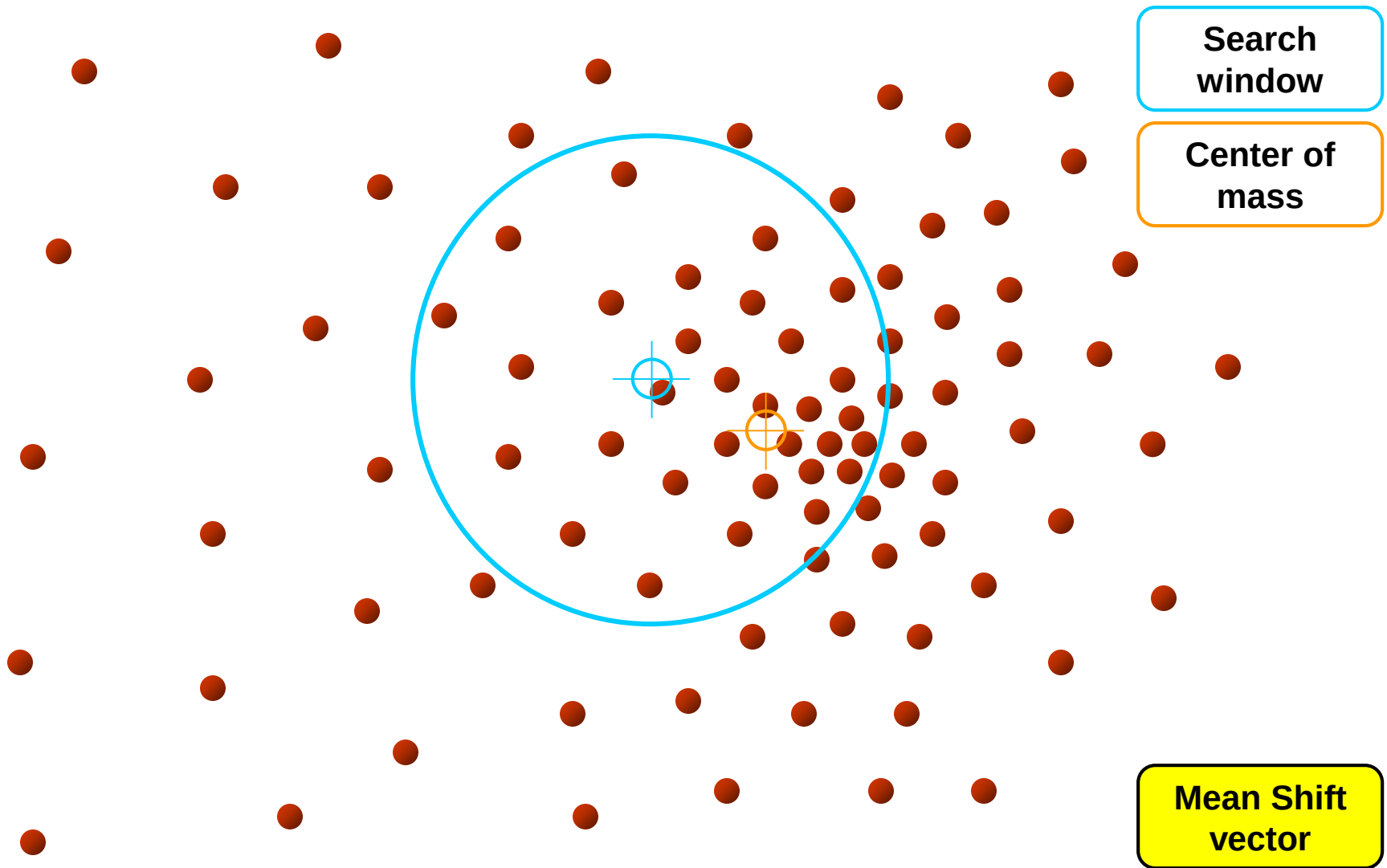
Mean shift



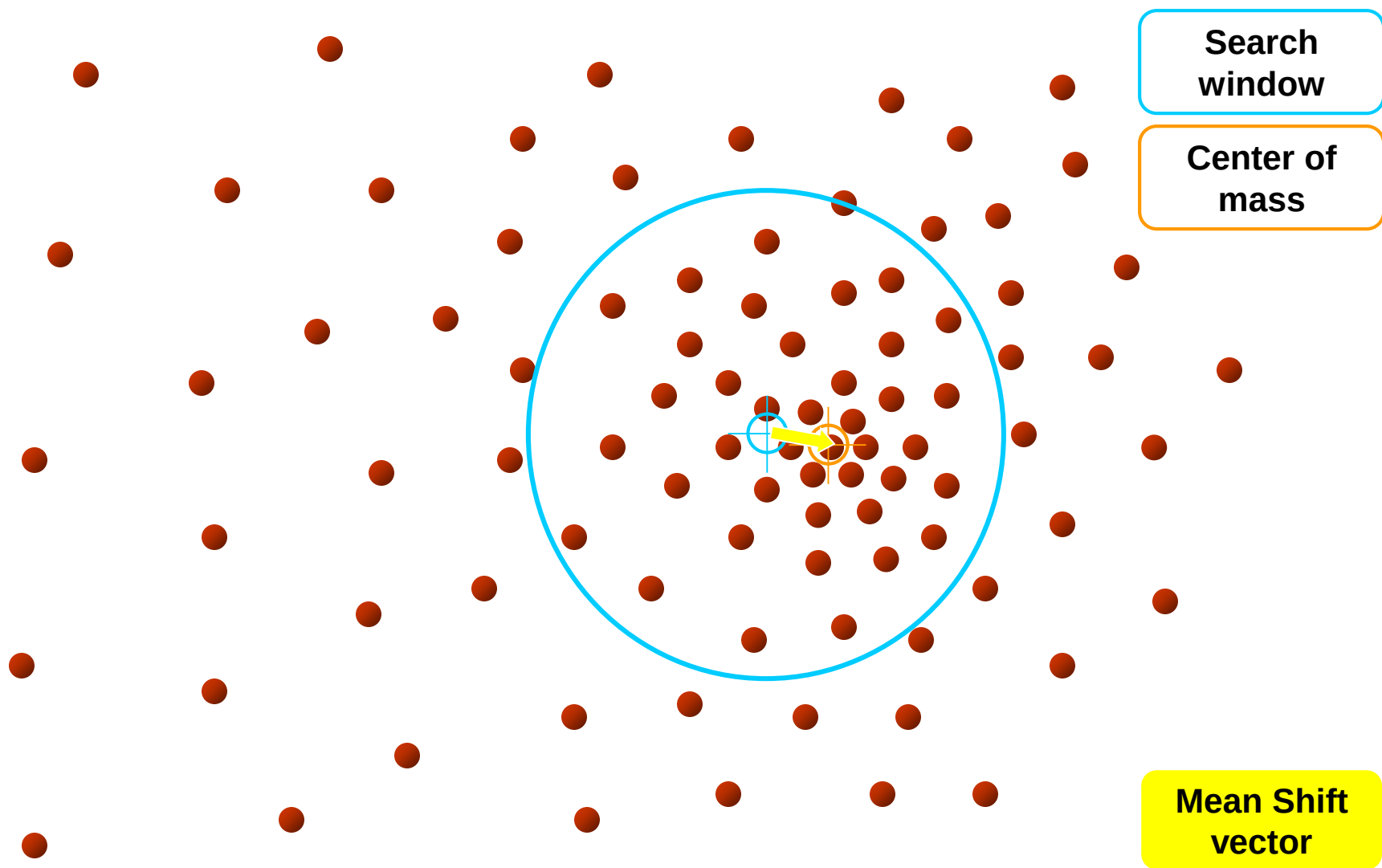
Mean shift



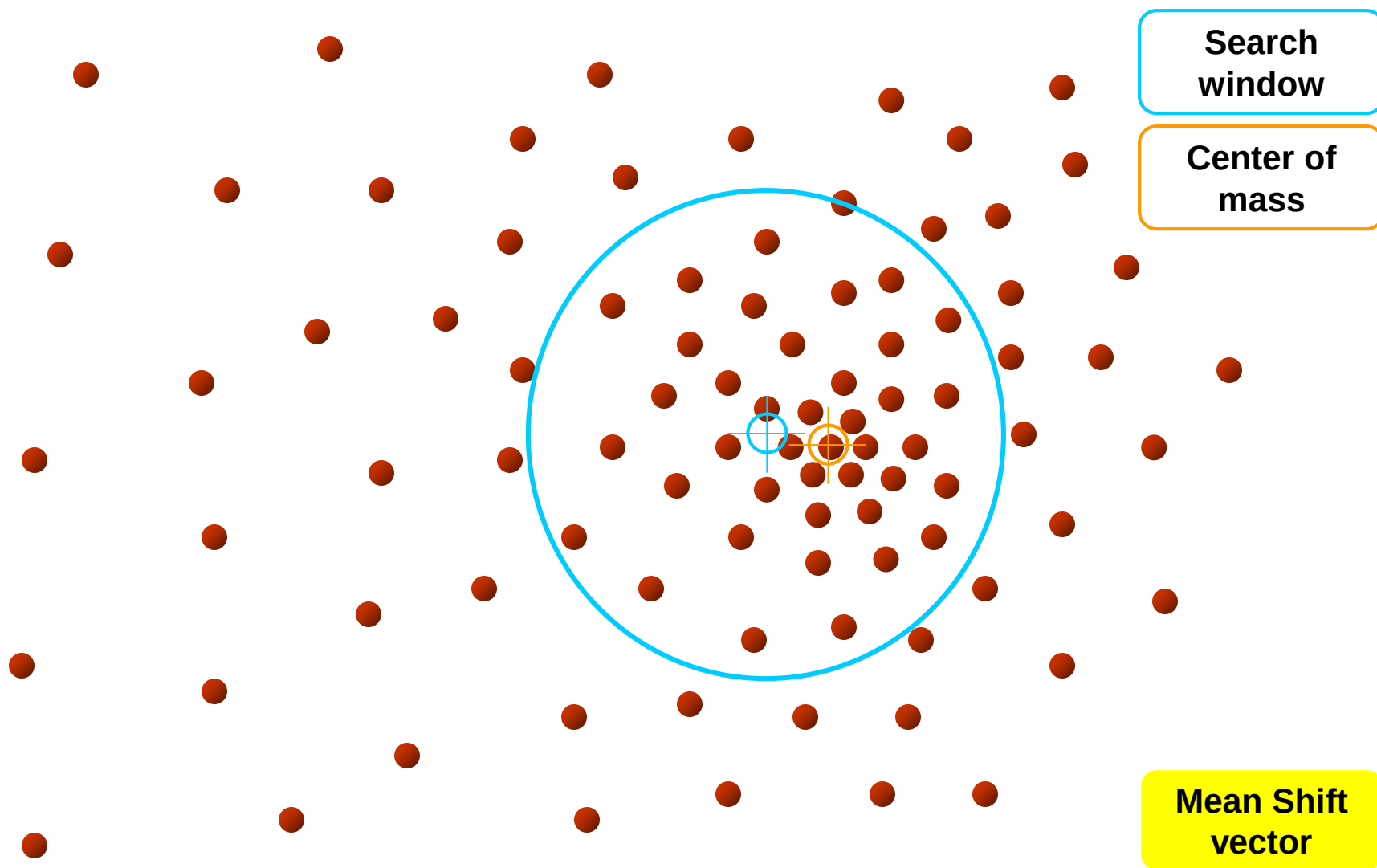
Mean shift



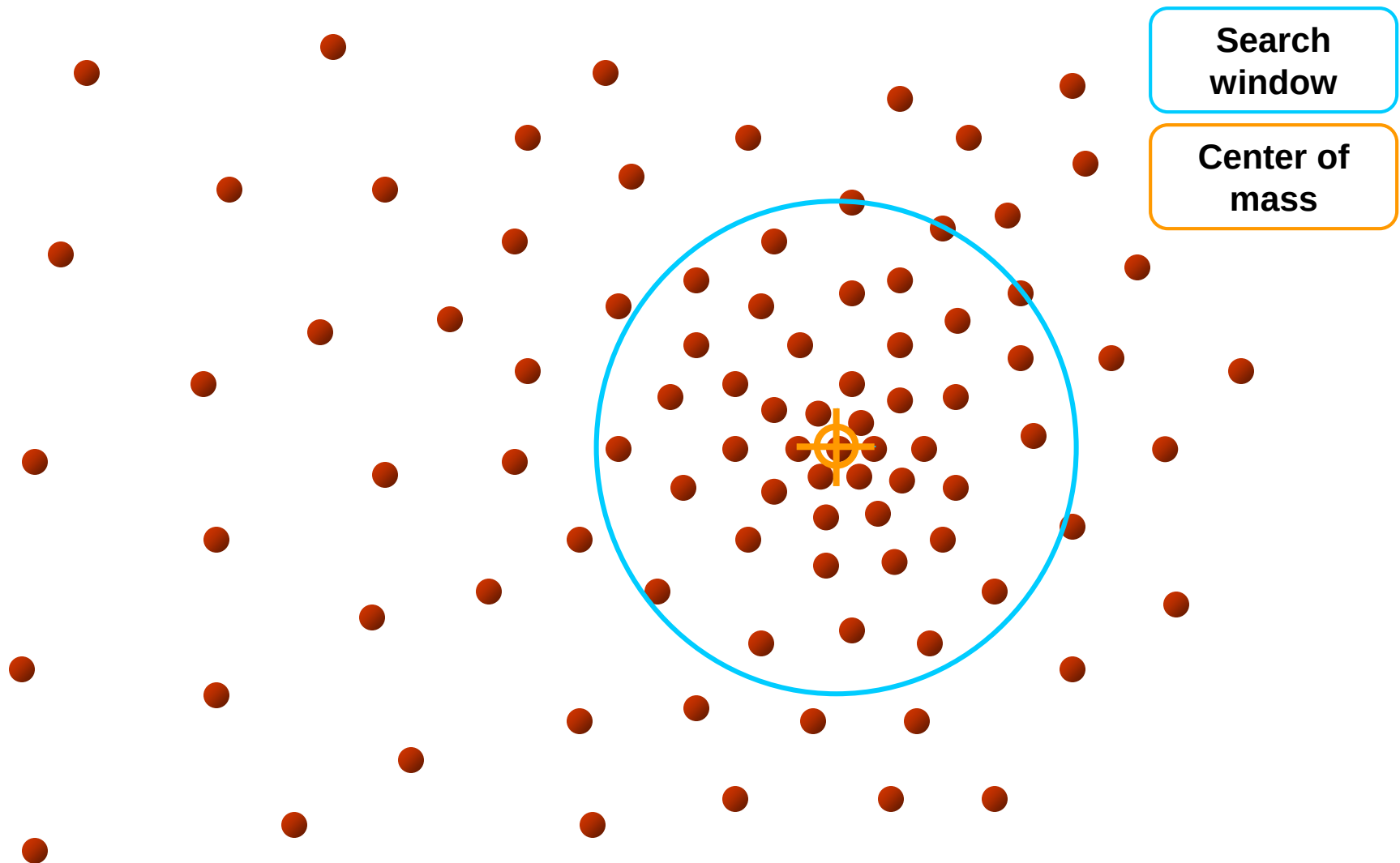
Mean shift



Mean shift

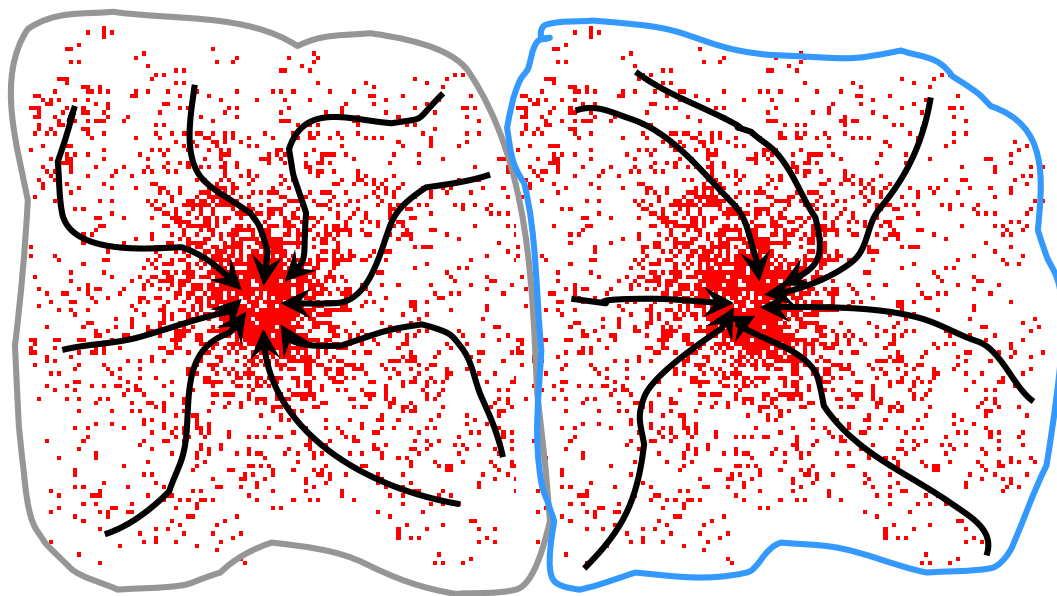


Mean shift



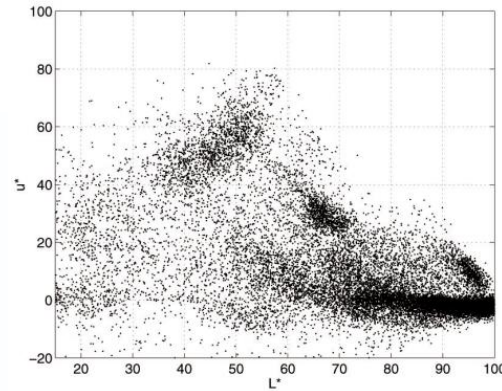
Mean shift clustering

- Cluster: all data points in the attraction basin of a mode
- Attraction basin: the region for which all trajectories lead to the same mode

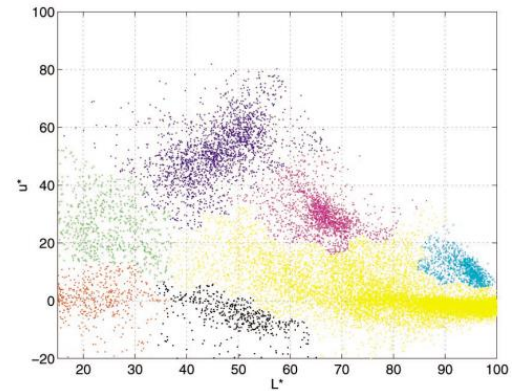


Mean shift clustering/segmentation

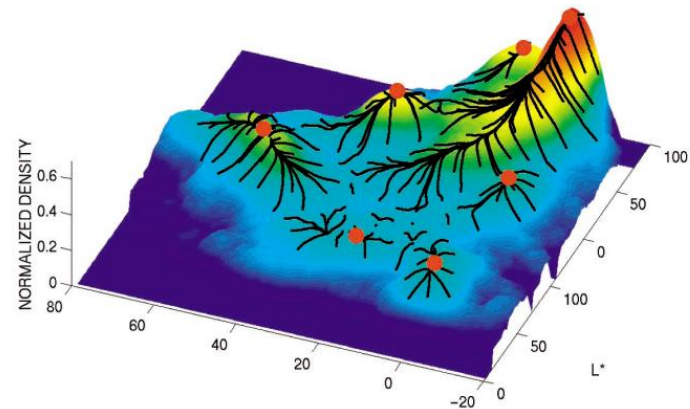
- Find features (color, gradients, texture, etc)
- Initialize windows at individual feature points
- Perform mean shift for each window until convergence
- Merge windows that end up near the same “peak” or mode



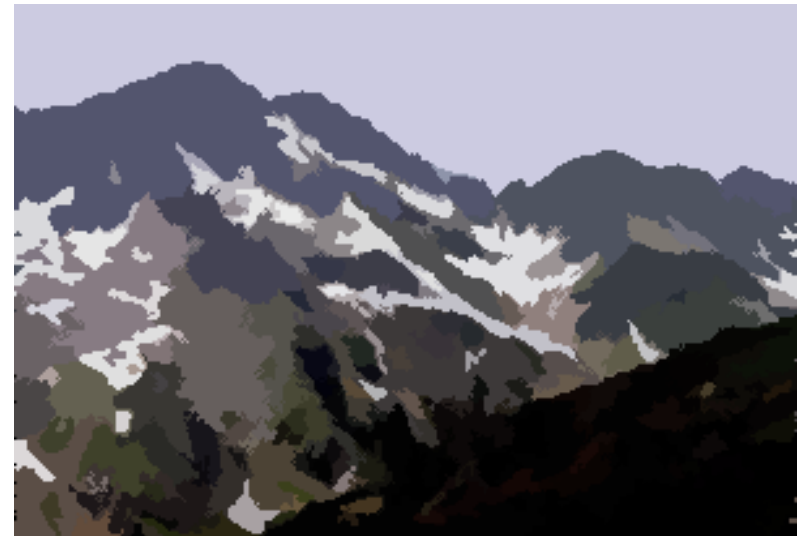
(a)



(b)



Mean shift segmentation results



<http://www.caip.rutgers.edu/~comanici/MSPAMI/msPamiResults.html>

More results



More results

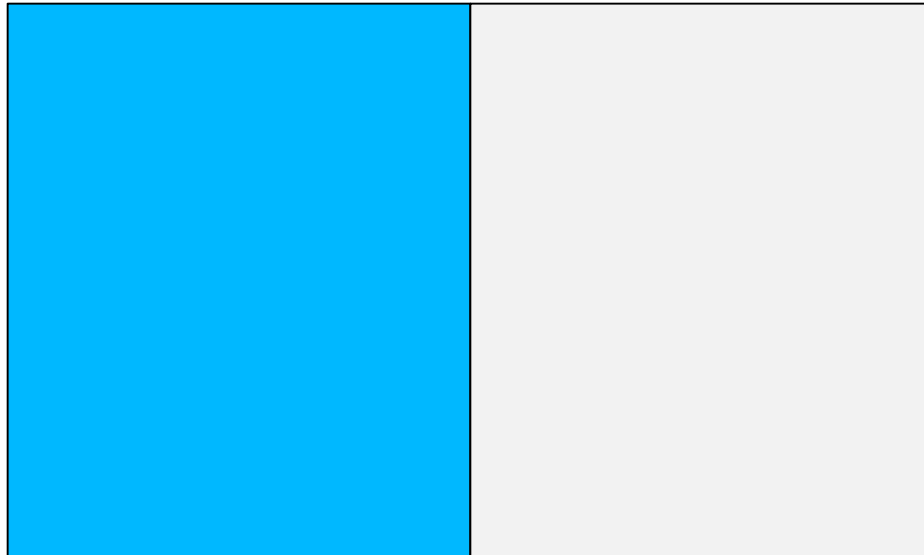


Mean shift pros and cons

- Pros
 - Does not assume spherical clusters
 - Just a single parameter (window size)
 - Finds variable number of modes
 - Robust to outliers
- Cons
 - Output depends on window size
 - Computationally expensive
 - Does not scale well with dimension of feature space

Normalized Cut

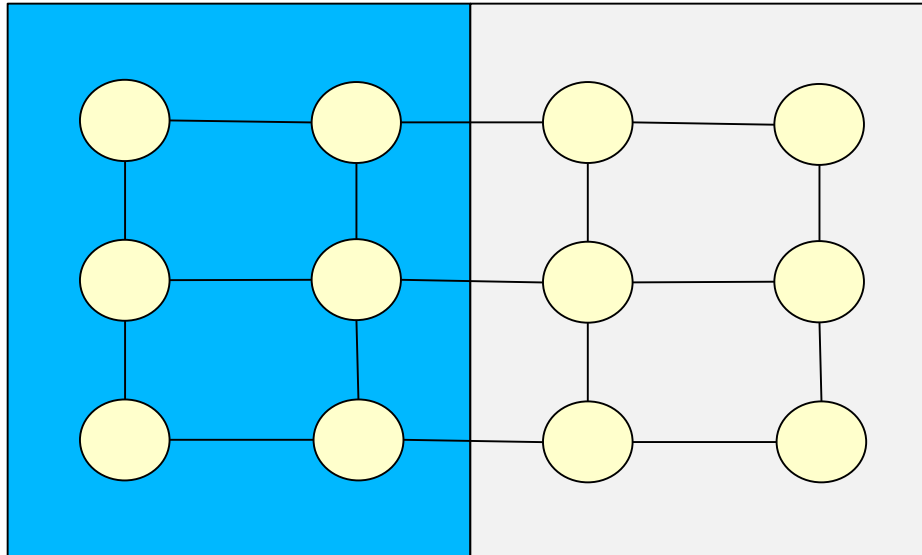
- How to cut the image below into two segments (by color)?



Normalized Cut

- How to cut the image below into two segments (by color)?

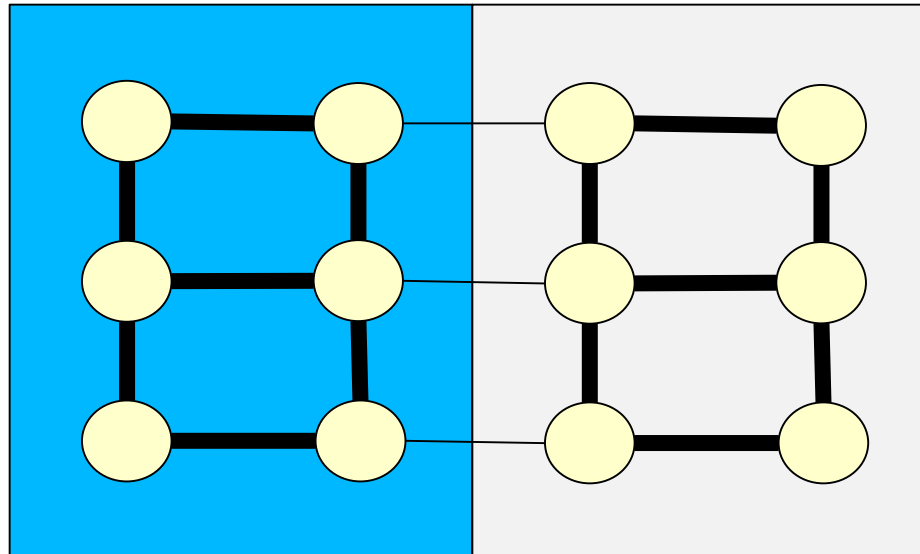
A Graphical View



Normalized Cut

- How to cut the image below into two segments (by color)?

A Graphical View

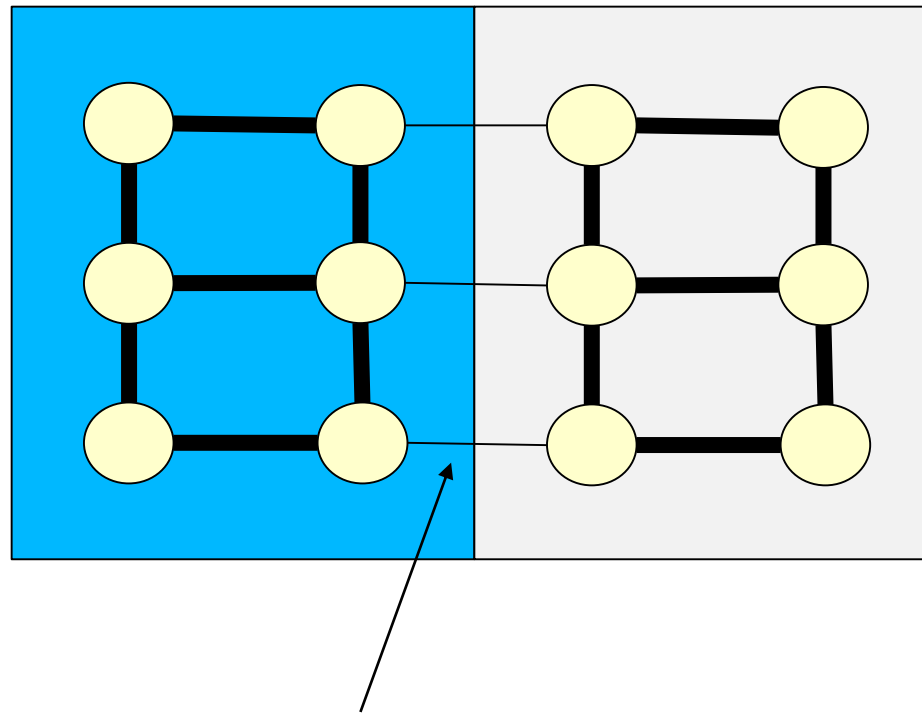


Let edge weight
correspond to the
similarity between
two adjacent nodes

Normalized Cut

- How to cut the image below into two segments (by color)?

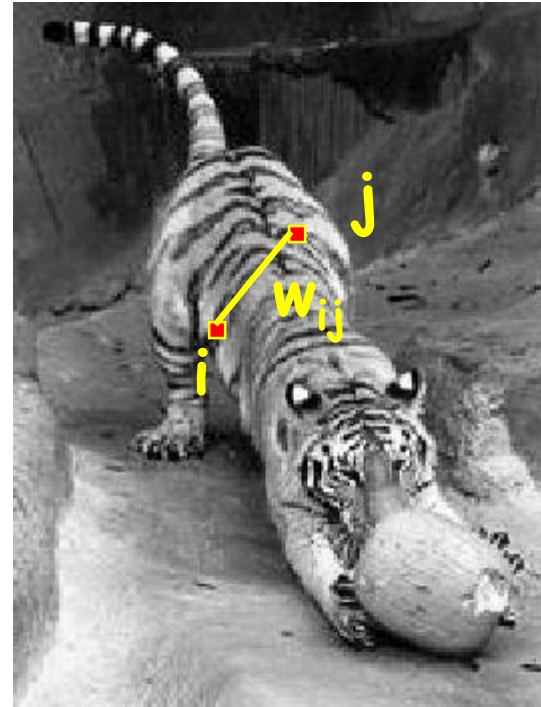
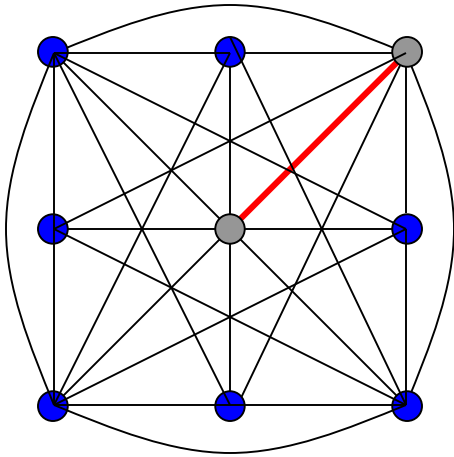
A Graphical View



Let edge weight
corresponds to the
similarity between
two adjacent nodes

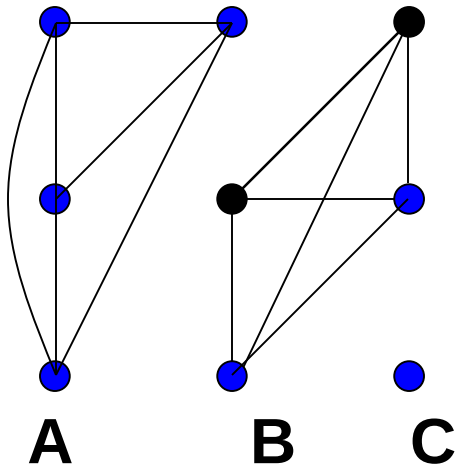
The optimal segmentation cuts through the thin edges

Images as graphs



- Node for every pixel
- Edge between every pair of pixels (or every pair of “sufficiently close” pixels)
- Each edge is weighted by the *affinity* or similarity of the two nodes

Segmentation by graph partitioning



- Break Graph into Segments
 - Delete links that cross between segments
 - Easiest to break links that have low affinity
 - similar pixels should be in the same segments
 - dissimilar pixels should be in different segments

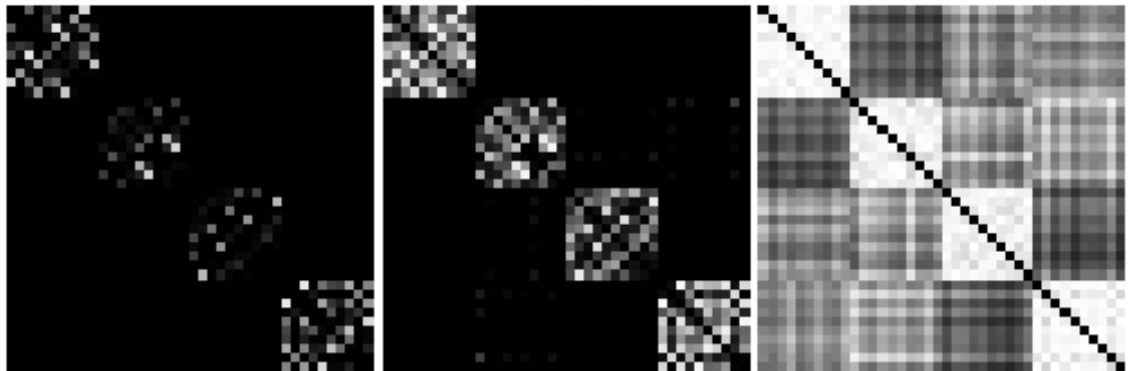
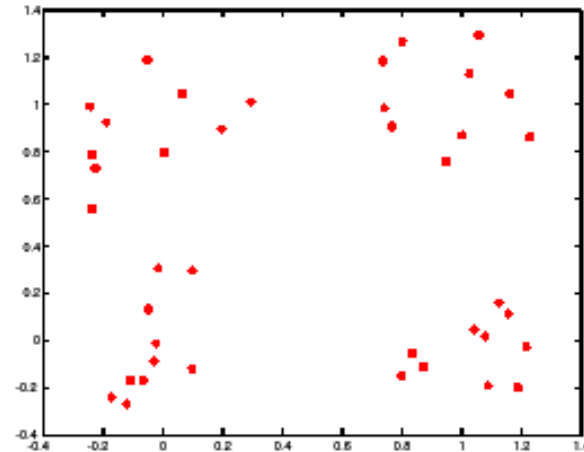
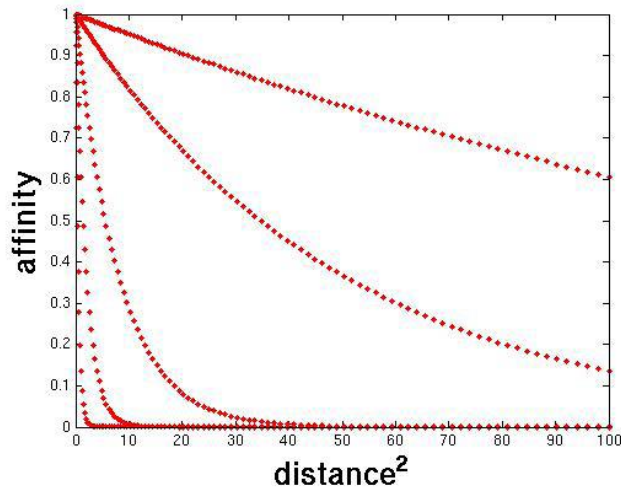
Measuring affinity

- Suppose we represent each pixel by a feature vector $\mathbf{x}$, and define a distance function appropriate for this feature representation
- Then we can convert the distance between two feature vectors into an affinity with the help of a generalized Gaussian kernel:

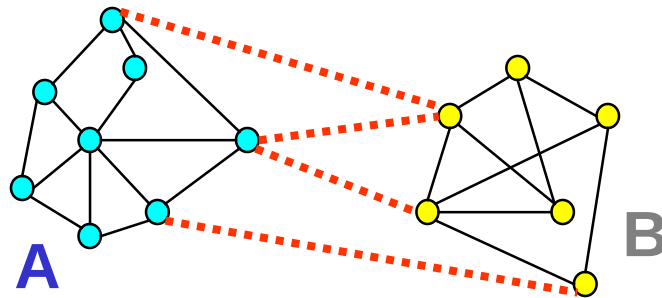
$$\exp\left(-\frac{1}{2\sigma^2} \text{dist}(\mathbf{x}_i, \mathbf{x}_j)^2\right)$$

Scale affects affinity

- Small σ : group only nearby points
- Large σ : group far-away points



Graph cut

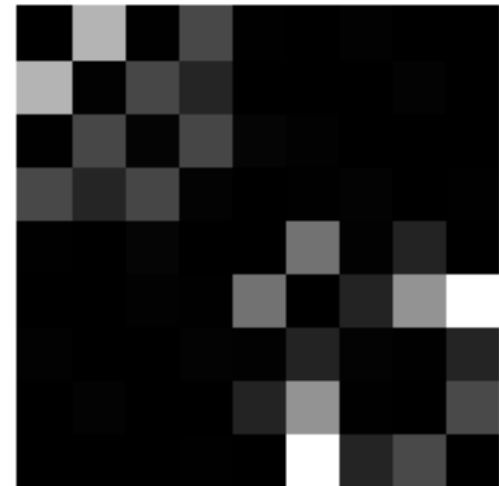
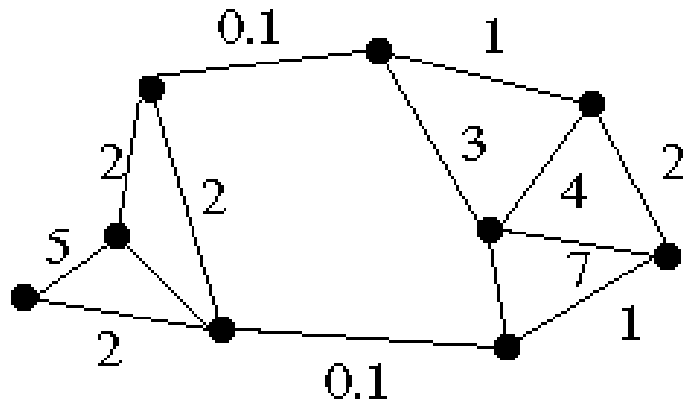


- Set of edges whose removal makes a graph disconnected
- Cost of a cut: sum of weights of cut edges
- A graph cut gives us a segmentation
 - What is a “good” graph cut and how do we find one?

Minimum cut

- We can do segmentation by finding the *minimum cut* in a graph
 - Efficient algorithms exist for doing this

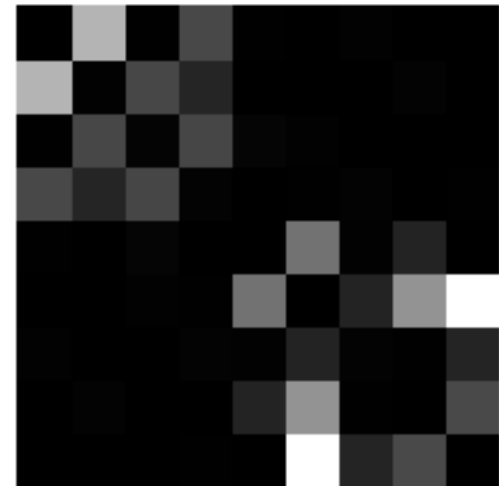
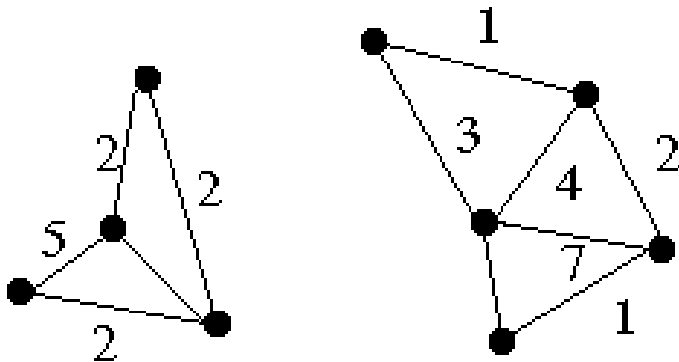
Minimum cut example



Minimum cut

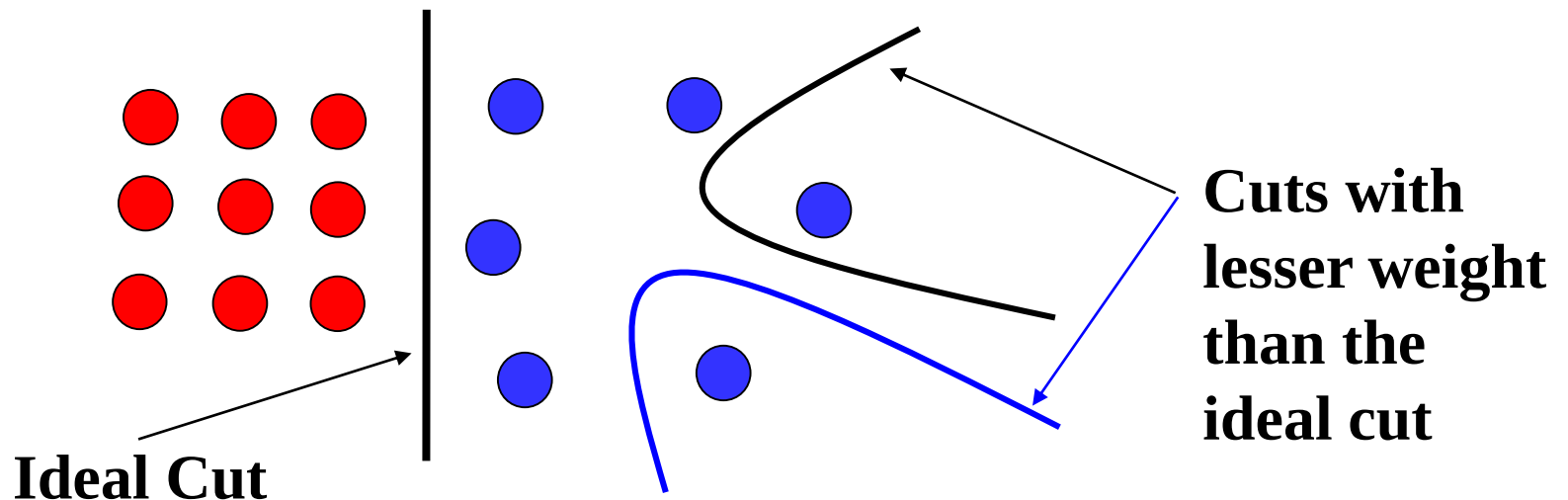
- We can do segmentation by finding the *minimum cut* in a graph
 - Efficient algorithms exist for doing this

Minimum cut example



Normalized cut

- Drawback: minimum cut tends to cut off very small, isolated components



Normalized cut

- Drawback: minimum cut tends to cut off very small, isolated components
- This can be fixed by normalizing the cut by the weight of all the edges incident to the segment
- The *normalized cut* cost is:

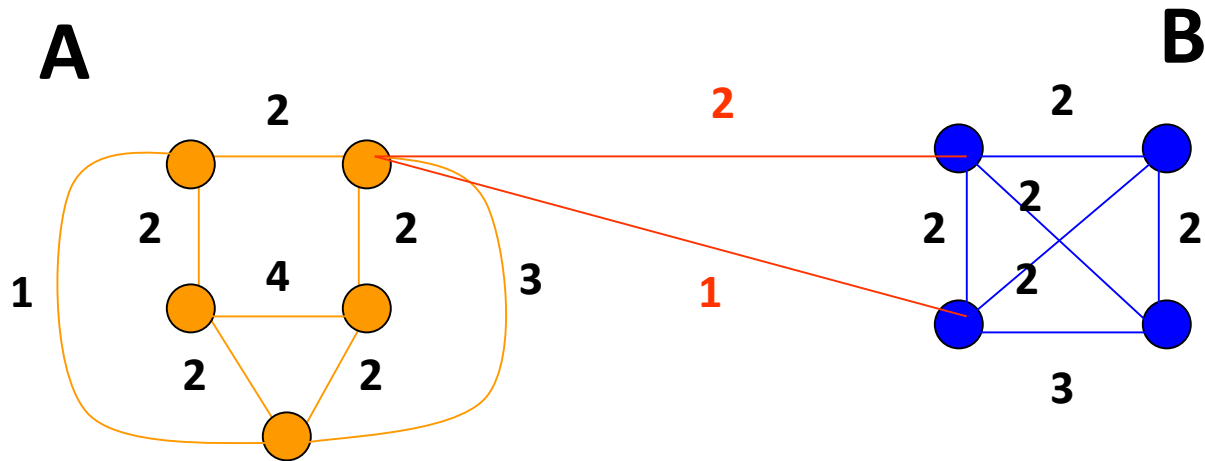
$$\frac{w(A, B)}{\text{assoc}(A, V)} + \frac{w(A, B)}{\text{assoc}(B, V)}$$

$w(A, B)$ = sum of weights of all edges between A and B

$\text{assoc}(A, V)$ = sum of all weights in cluster A + $w(A, B)$

$$\text{assoc}(A, V) = \sum_{u \in A, t \in V} w(u, t)$$

Example



$$\text{Ncut}(A,B) = \frac{3}{21} + \frac{3}{16}$$

Normalized cut

- Finding the exact minimum of the normalized cut cost is NP-complete, but we *relax* to let nodes take on arbitrary values:
- Let W be the adjacency matrix of the graph
- Let D be the diagonal matrix with diagonal entries $D(i, i) = \sum_j W(i, j)$
- Then the normalized cut cost can be written as

$$\frac{y^T (D - W) y}{y^T D y}$$

where y is an indicator vector whose value should be 1 in the i th position if the i th feature point belongs to A and a negative constant otherwise

Normalized cut

- We can minimize the relaxed cost by solving the *generalized eigenvalue problem* $(D - W)y = \lambda Dy$
- The solution y is given by the generalized eigenvector corresponding to the second smallest eigenvalue
- Intuitively, the i th entry of y can be viewed as a “soft” indication of the component membership of the i th feature
 - Can use 0 or median value of the entries as the splitting point (threshold), or find threshold that minimizes the Ncut cost

Normalized cut algorithm

1. Represent the image as a weighted graph $G = (V, E)$, compute the weight of each edge, and summarize the information in D and W
2. Solve $(D - W)y = \lambda Dy$ for the eigenvector with the second smallest eigenvalue
3. Use the entries of the eigenvector to bipartition the graph
 - To find more than two clusters:
 - Recursively bipartition the graph
 - Run k-means clustering on values of several eigenvectors

Normalize Cut in Matrix Form

W is the cost matrix : $\mathbf{W}(i, j) = w_{i,j}$;

D is the sum of costs from node i : $\mathbf{D}(i, i) = \sum_j \mathbf{W}(i, j)$;

After lots of math, we get:

$$Ncut(A, B) = \frac{\mathbf{y}^T (\mathbf{D} - \mathbf{W}) \mathbf{y}}{\mathbf{y}^T \mathbf{D} \mathbf{y}}, \quad \text{with } y_i \in \{1, -1\}, \mathbf{y}^T \mathbf{D} \mathbf{1} = 0.$$

- Solution given by "generalized" eigenvalue problem:

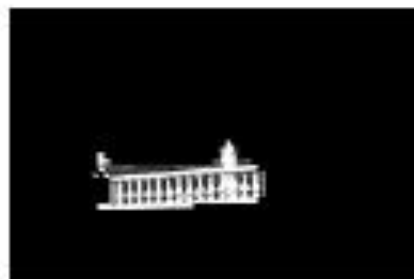
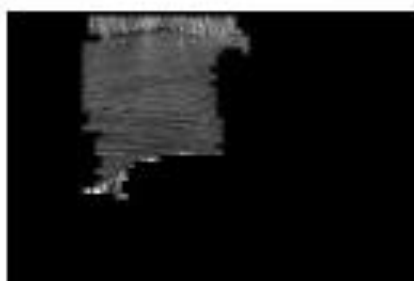
$$(\mathbf{D} - \mathbf{W}) \mathbf{y} = \lambda \mathbf{D} \mathbf{y}$$

- Solved by converting to standard eigenvalue problem:

$$\mathbf{D}^{-\frac{1}{2}} (\mathbf{D} - \mathbf{W}) \mathbf{D}^{-\frac{1}{2}} \mathbf{z} = \lambda \mathbf{z}, \quad \text{where } \mathbf{z} = \mathbf{D}^{\frac{1}{2}} \mathbf{y}$$

- optimal solution corresponds to second smallest eigenvector
- for more details, see
 - J. Shi and J. Malik, [Normalized Cuts and Image Segmentation](#), IEEE Conf. Computer Vision and Pattern Recognition(CVPR), 1997

Example result



Normalized cuts: Pro and con

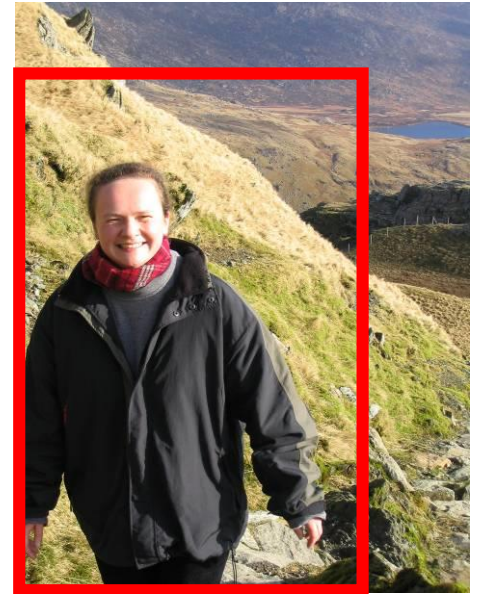
- Pros
 - Generic framework, can be used with many different features and affinity formulations
- Cons
 - High storage requirement and time complexity
 - Bias towards partitioning into equal segments

What is Missing in Normalized Cut?

Global Model for Object/Background!

- Where to obtain the global appearance information
- How to build the object/background model

Interactive Object Cutout and Composition



Graph Cut for Image Segmentation

User's Stroke (Blue for “object”,
red for “background”)

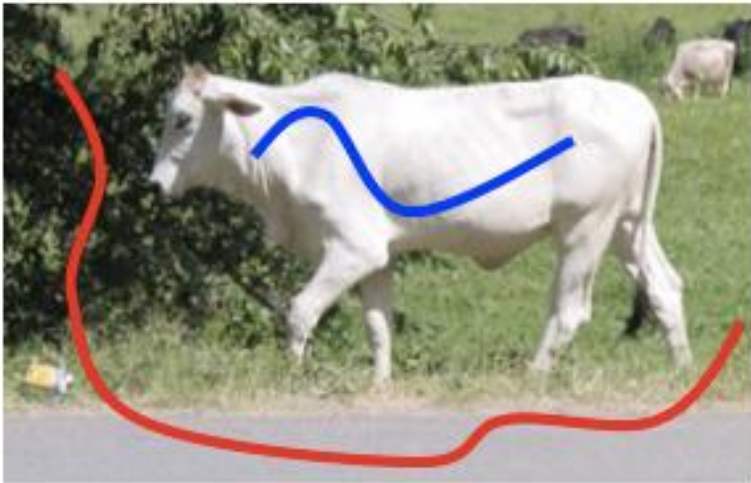
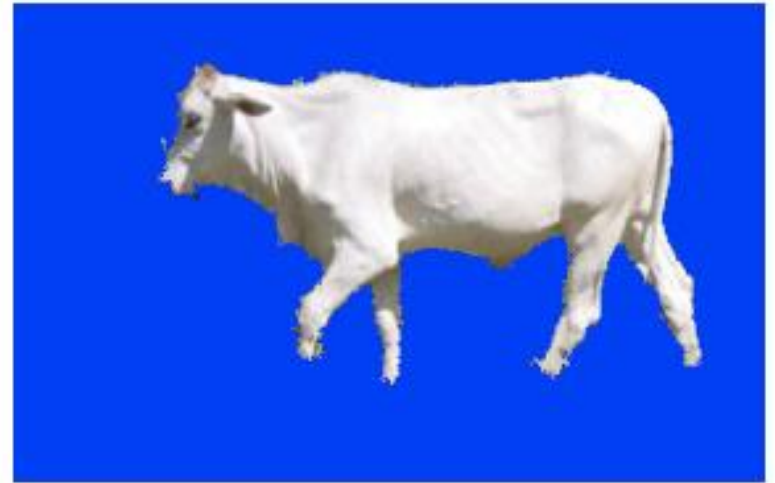
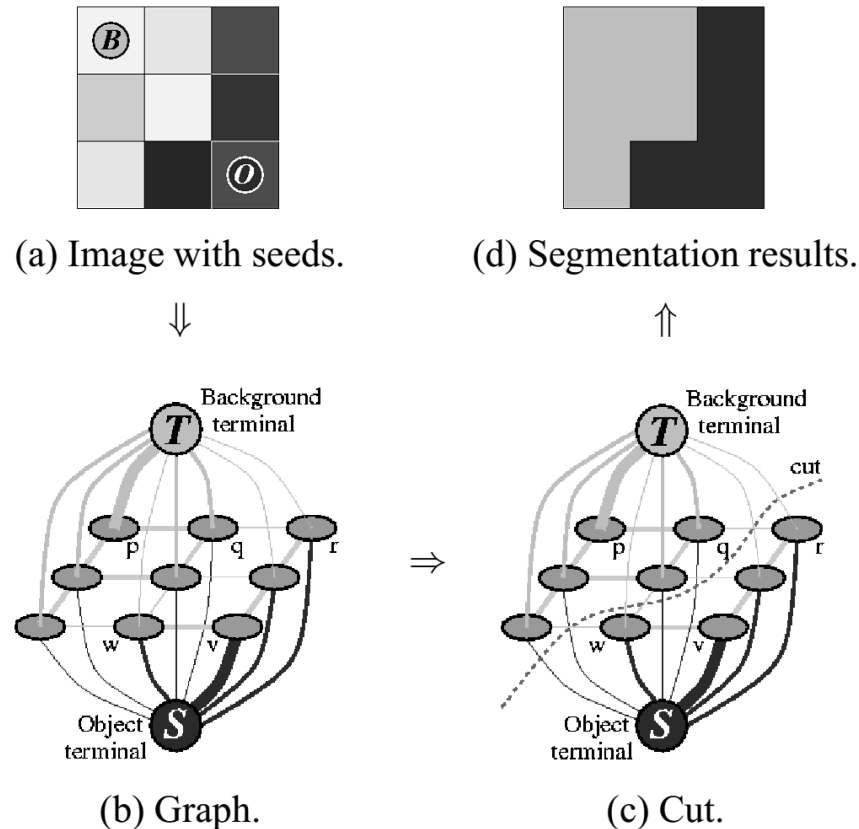


Image segmentation using Graph
Cut



Yuri Boykov, Marie-Pierre Jolly, Interactive Graph Cuts for Optimal Boundary & Region Segmentation of Objects in N-D Images, ICCV 2001

Graph Cut for Image Segmentation

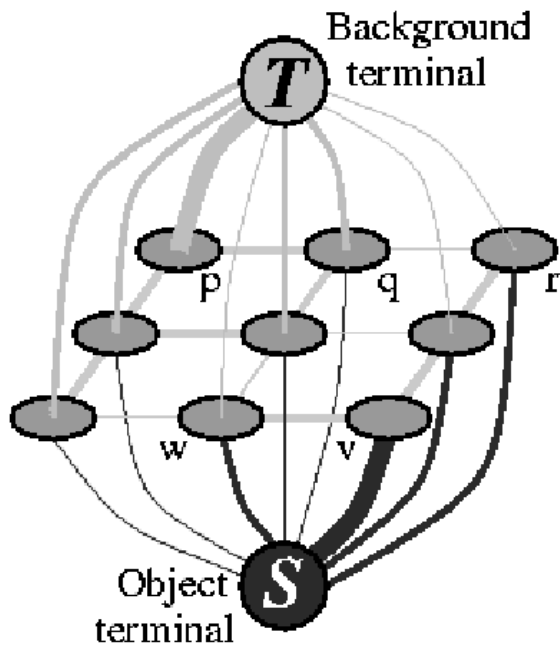


**Cast interactive image
segmentation as a Graph Cut
problem**

Figure 1. A simple 2D segmentation example for a 3×3 image. The seeds are $\mathcal{O} = \{v\}$ and $\mathcal{B} = \{p\}$. The cost of each edge is reflected by the edge's thickness. The regional term (2) and hard constraints (4,5) define the costs of t-links. The boundary term (3) defines the costs of n-links. Inexpensive edges are attractive choices for the minimum cost cut.

Graph Cut for Image Segmentation

- The weights of edges



edge	weight (cost)	for
$\{p, q\}$	$B_{\{p,q\}}$	$\{p, q\} \in \mathcal{N}$
$\{p, S\}$	$\lambda \cdot R_p(\text{"bkg"})$	$p \in \mathcal{P}, p \notin \mathcal{O} \cup \mathcal{B}$
	K	$p \in \mathcal{O}$
	0	$p \in \mathcal{B}$
$\{p, T\}$	$\lambda \cdot R_p(\text{"obj"})$	$p \in \mathcal{P}, p \notin \mathcal{O} \cup \mathcal{B}$
	0	$p \in \mathcal{O}$
	K	$p \in \mathcal{B}$

Graph Cut for Image Segmentation

- Use histogram to define the “regional” (or unary) term

$$\begin{aligned} R_p(\text{“obj”}) &= -\ln \Pr(I_p | \mathcal{O}) \\ R_p(\text{“bkg”}) &= -\ln \Pr(I_p | \mathcal{B}). \end{aligned}$$

- The binary term enforces “smoothness” among similar pixels

$$B_{\{p,q\}} \propto \exp\left(-\frac{(I_p - I_q)^2}{2\sigma^2}\right) \cdot \frac{1}{\text{dist}(p,q)}.$$

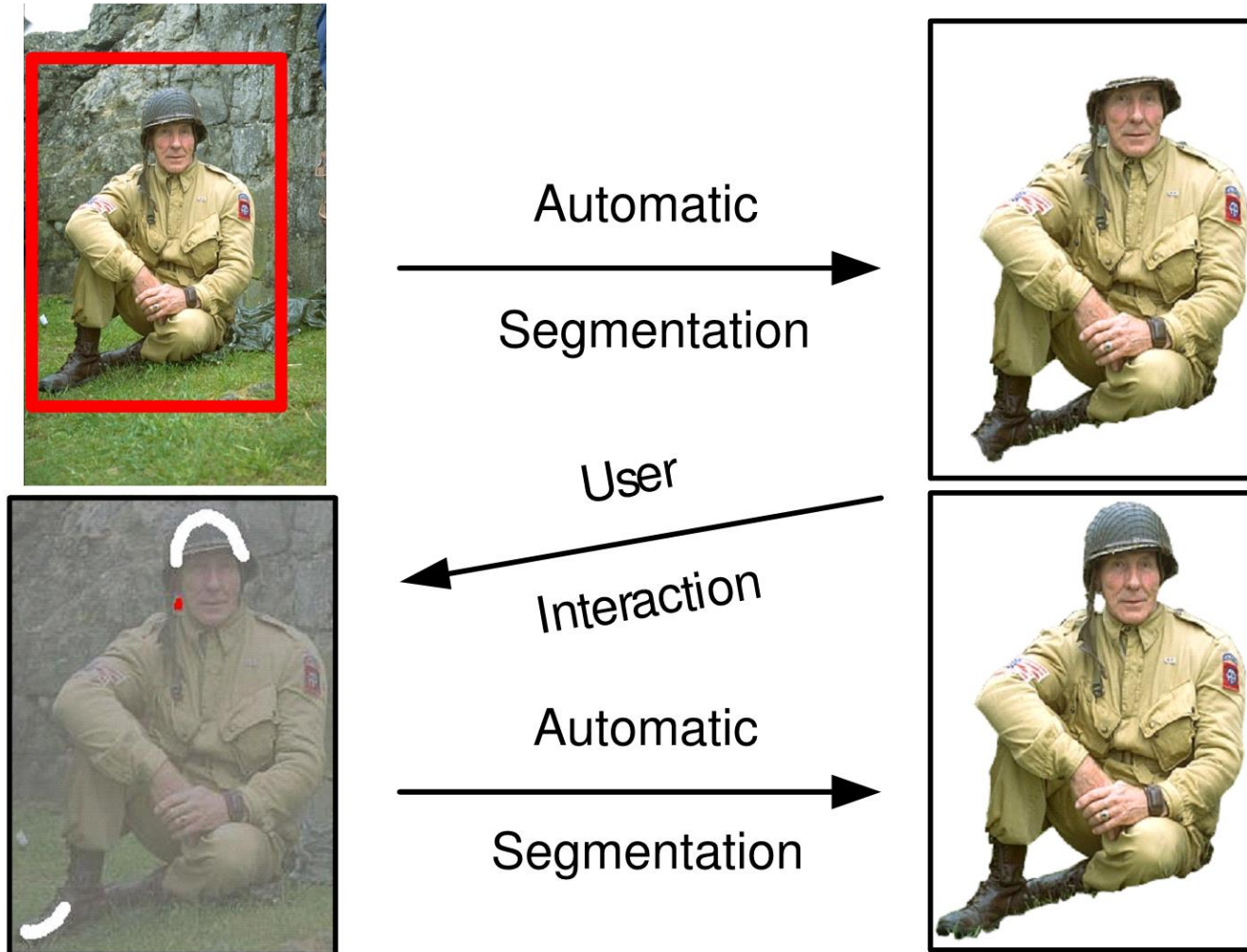
Color similarity

Spatial adjacency

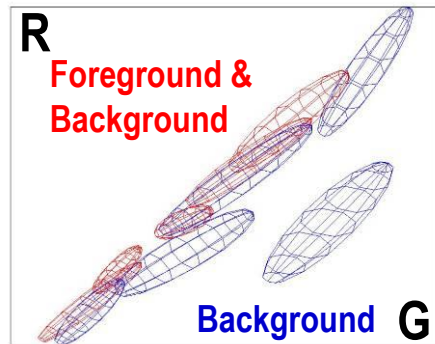
Graph Cut for Image Segmentation

- The general formulation for graph cut
 - Binary-variable energy minimization
- The solver
 - Max-flow algorithm

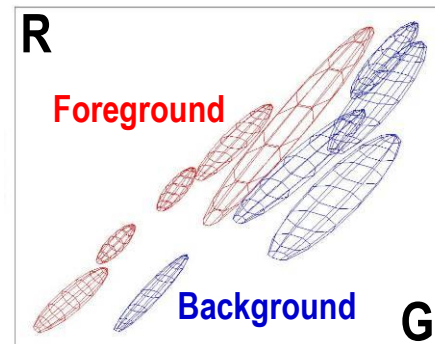
GrabCut



GMM Color Model



Iterated graph
cut



Gaussian Mixture Model (typically 5-8 components)

Initialisation

- User initialises trimap T by supplying only T_B . The foreground is set to $T_F = \emptyset$; $T_U = \overline{T_B}$, complement of the background.
- Initialise $\alpha_n = 0$ for $n \in T_B$ and $\alpha_n = 1$ for $n \in T_U$.
- Background and foreground GMMs initialised from sets $\alpha_n = 0$ and $\alpha_n = 1$ respectively.

Iterative minimisation

1. *Assign GMM components to pixels:* for each n in T_U ,

$$k_n := \arg \min_{k_n} D_n(\alpha_n, k_n, \theta, z_n).$$

2. *Learn GMM parameters* from data $\mathbf{z}$:

$$\underline{\theta} := \arg \min_{\underline{\theta}} U(\underline{\alpha}, \mathbf{k}, \underline{\theta}, \mathbf{z})$$

3. *Estimate segmentation:* use min cut to solve:

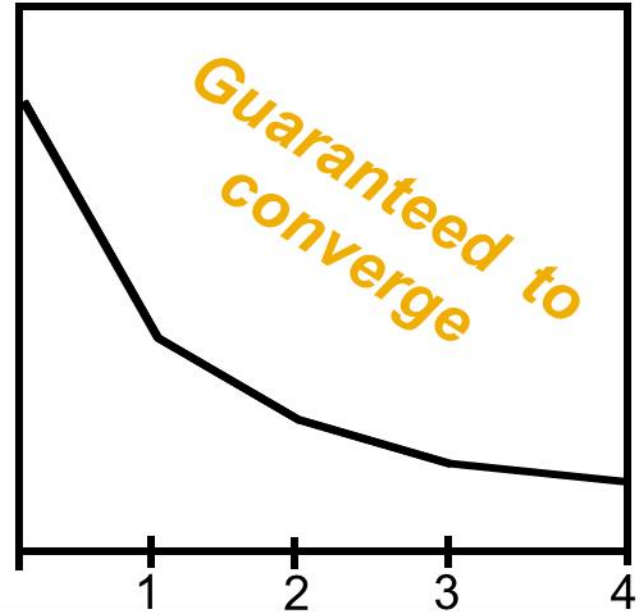
$$\min_{\{\alpha_n: n \in T_U\}} \min_{\mathbf{k}} \mathbf{E}(\underline{\alpha}, \mathbf{k}, \underline{\theta}, \mathbf{z}).$$

4. Repeat from step 1, until convergence.
5. Apply border matting (section 4).

Iterated Graph Cut



Result



Energy after each Iteration

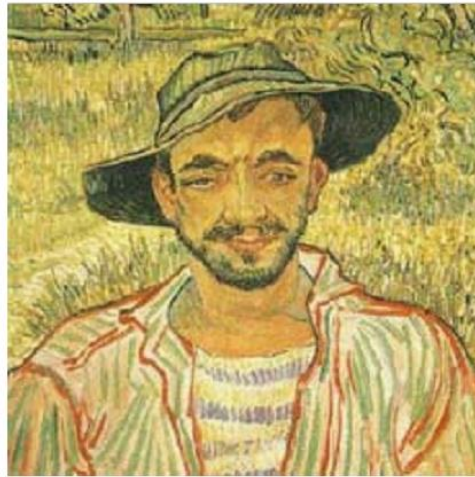
GrabCut



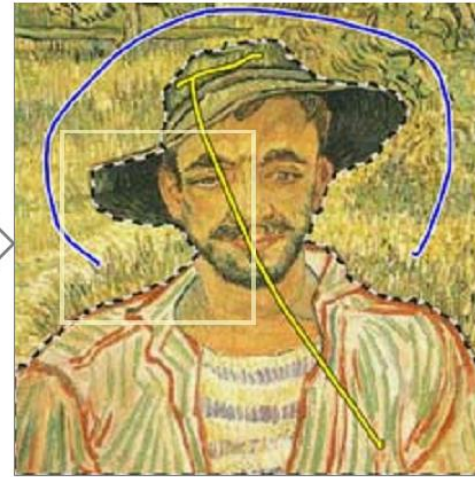
No User
Interaction



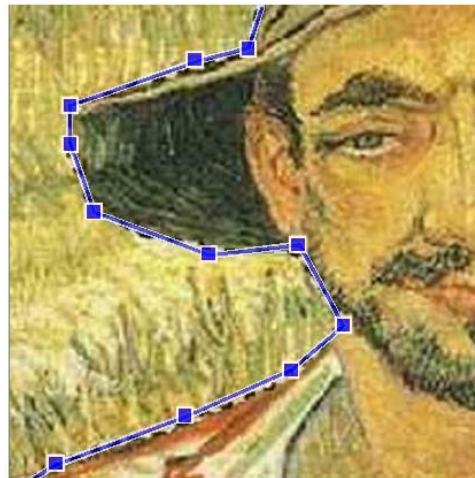
LazySnapping



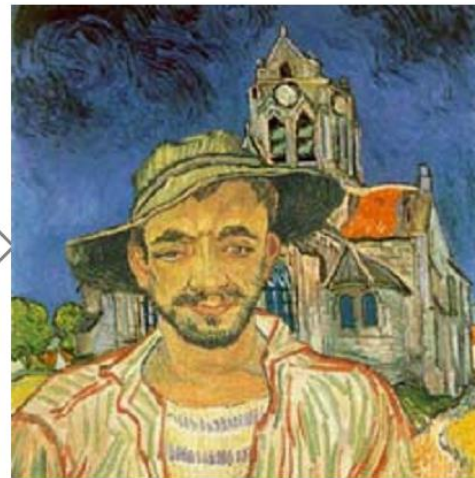
(a) Input image



(b) Object Marking



(c) Boundary editing



(d) Output composition

Yin Li et al., Lazy Snapping, SIGGRAPH 2004

Lazy Snapping Overview

- Main Steps:

1. Mark strokes as foreground & background
2. Perform Boykov & Jolly style graphcut
3. *Edit Boundary*

- Familiar Formulation

- Minimize Gibbs Energy:

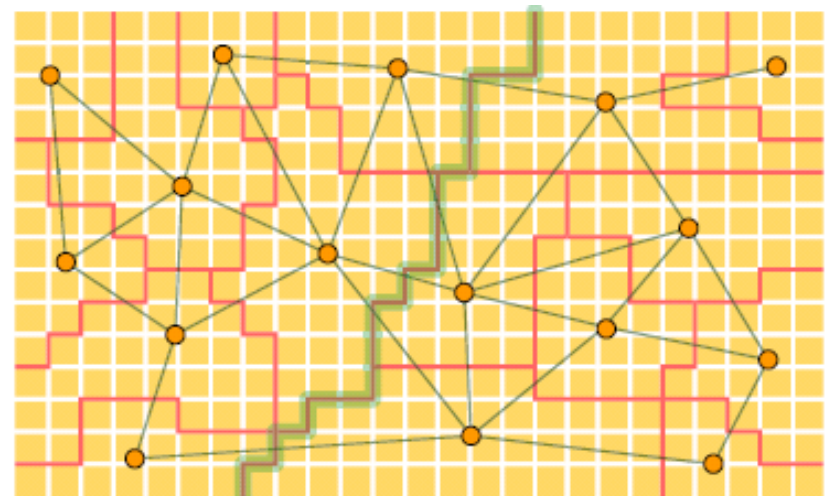
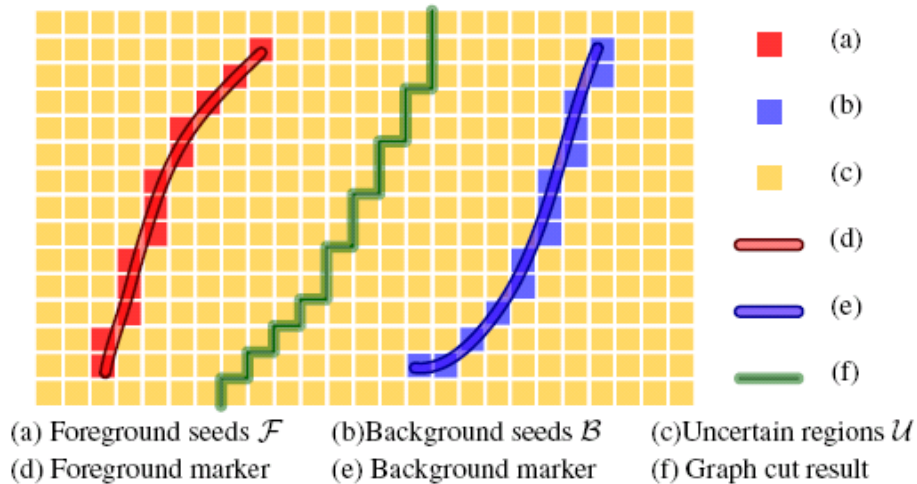
$$E(X) = \sum_{i \in \mathcal{V}} E_1(x_i) + \lambda \sum_{(i,j) \in \mathcal{E}} E_2(x_i, x_j)$$

- $E_1(X)$ – Cluster (K-means) known foreground/background colors to find representative colors
- $E_2(X)$ – Similar to Boykov & Jolly

Lazy Snapping – Issue

- Too Slow to allow for real-time refinement
- Solution?

Superpixels!



Lazy Snaping – Speedup

Image	Dimension	Nodes Ratio	Edges Ratio	Lag with Pre-segmentation	Lag without Pre-segmentation
Boy	(408, 600)	10.7	16.8	0.12s	0.57s
Ballet	(440, 800)	11.4	18.3	0.21s	1.39s
Twins	(1024, 768)	20.7	32.5	0.25s	1.82s
Girl	(768, 1147)	23.8	37.6	0.22s	2.49s
Grandpa	(1147, 768)	19.3	30.5	0.22s	3.56s

More Results



(a) Girl (4/2/12)



(b) Ballet (4/7/14)



(c) Boy (6/2/13)



(e) Grandpa (4/2/11)



(d) Twins (4/4/12)



Matting and compositing



The matting equations

$$I_i = \alpha_i F_i + (1 - \alpha_i) B_i$$



$$I_i = \alpha_i F_i + (1 - \alpha_i) B_i$$

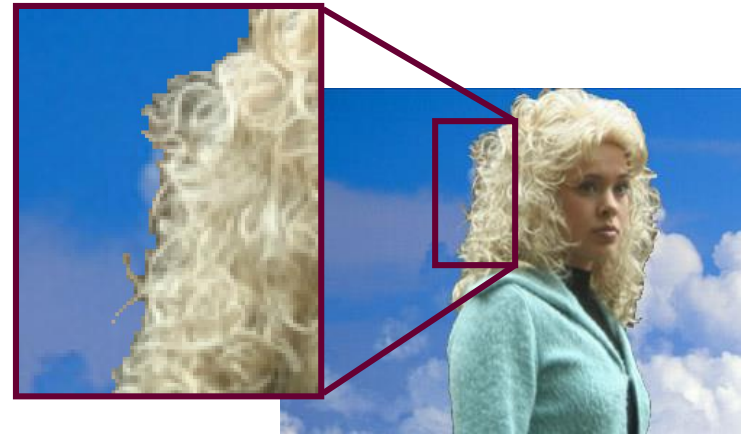




Source image



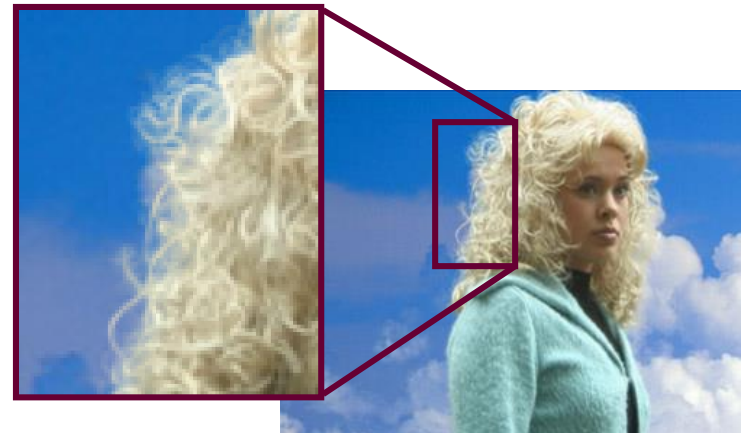
Hard segmentation



compositing

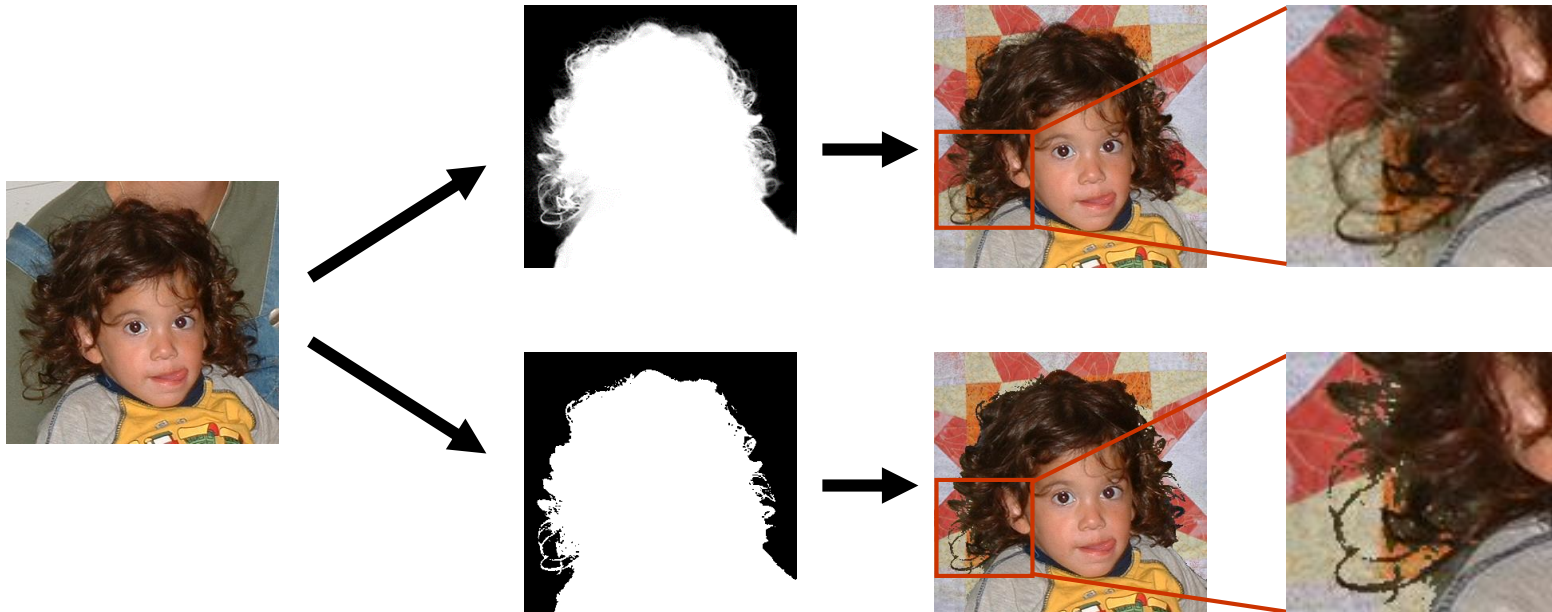


matte



compositing

Why is matting hard?



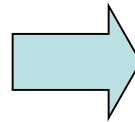
Why is matting hard?



Why is matting hard?



7 Unknowns, 3 knowns



Observed
Image

Alpha Matte

$$I_z = \alpha_z F_z + (1 - \alpha_z) B_z$$

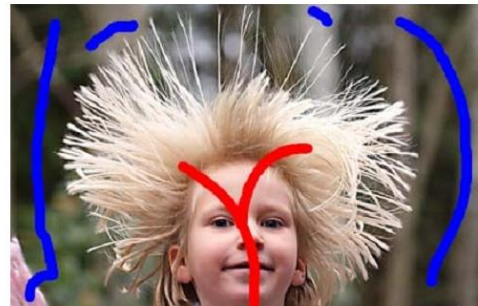
Diagram illustrating the equation $I_z = \alpha_z F_z + (1 - \alpha_z) B_z$ with labels:

- I_z is labeled **Known** (indicated by a red arrow).
- F_z is labeled **Unknown** (indicated by a blue arrow).
- α_z is labeled **Unknown** (indicated by a blue arrow).
- B_z is labeled **Unknown** (indicated by a blue arrow).

- Two main ways to make the matting problem tractable



Known background (Chroma keying)
- Image from Wikipedia



Scribble-based interface

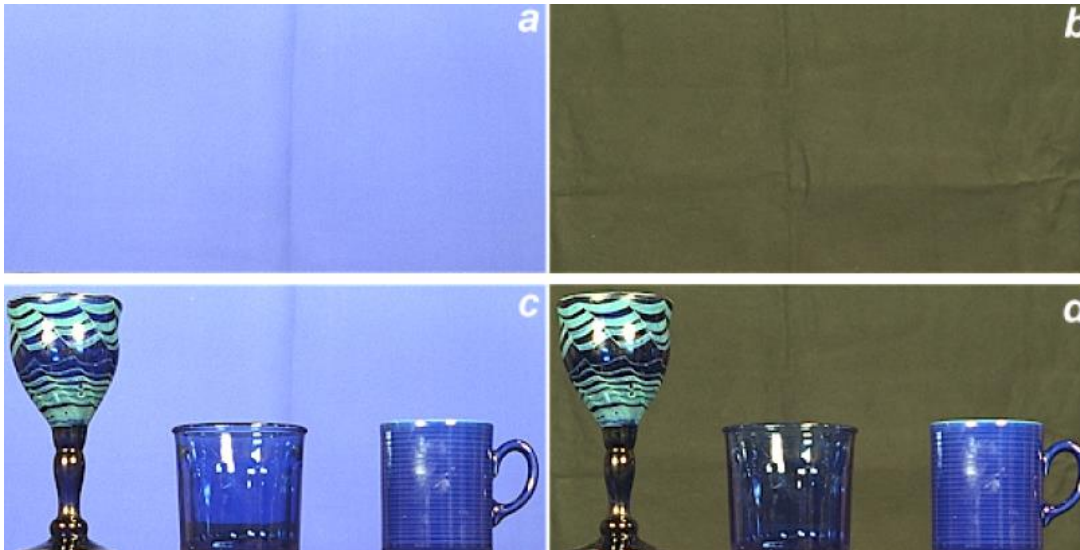


Trimap



Alpha matte
(BMVC08)

User interaction (natural image matting)

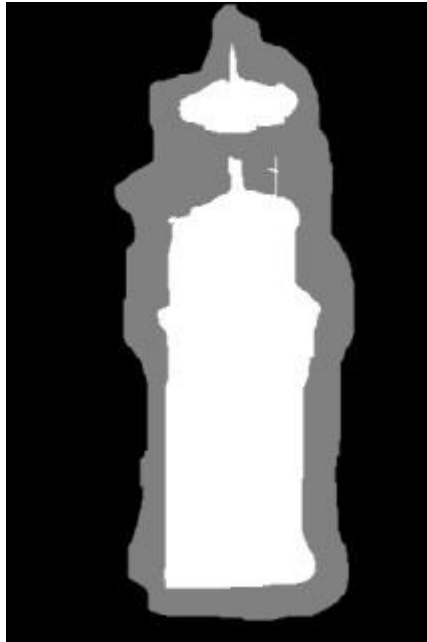


→ Backgrounds are estimated

→ Unknown:
 $\{\alpha, F_r, F_g, F_b\}$

We have 6 equations for 3 color channels in 2 images, and 4 unknowns.

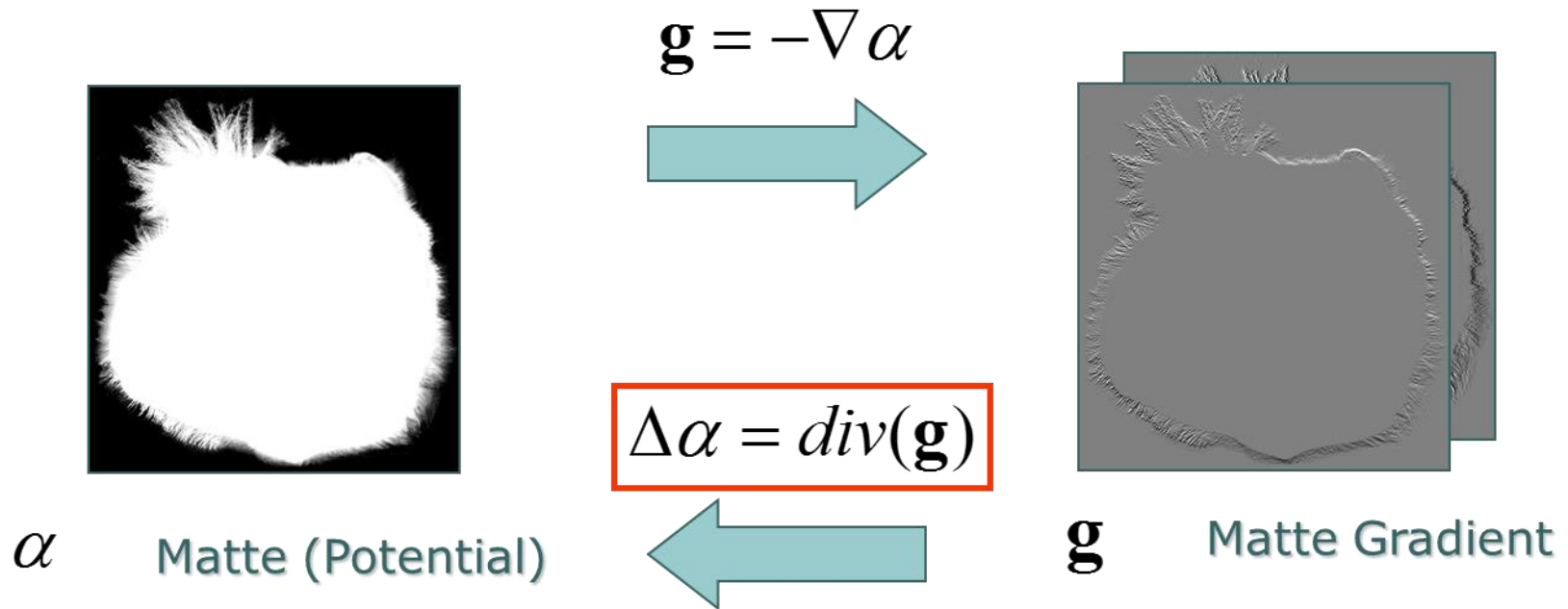
Poisson Matting



Jian Sun et al., Poisson Matting, SIGGRAPH 2004

Poisson Matting

- Solve the matting problem in gradient domain



$$\Delta \equiv \frac{\partial^2}{\partial^2 x} + \frac{\partial^2}{\partial^2 y}$$

$$\text{div}(\mathbf{g}) \equiv \frac{\partial g_x}{\partial x} + \frac{\partial g_y}{\partial y}$$

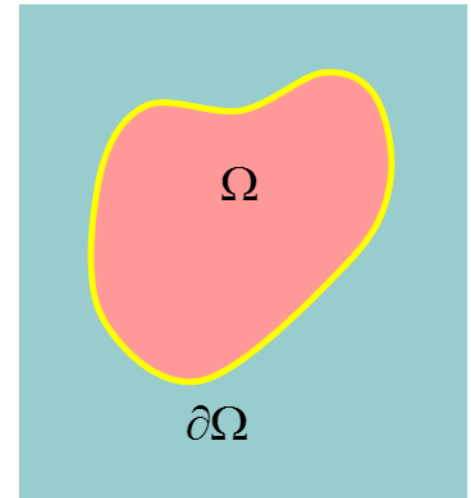
Poisson Matting

Given the destination matte gradient v , the optimization problem becomes:

$$\min_{\alpha} \int \int_{\Omega} | \nabla \alpha - v |^2 \text{ with } \alpha|_{\partial\Omega} = \alpha^*|_{\partial\Omega}$$



$$\Delta \alpha = \text{div}(\mathbf{g}) \quad s.t. \quad \alpha|_{\partial\Omega} = \alpha^*|_{\partial\Omega}$$



Poisson Matting

- From matting equation:

$$I = \alpha F + (1 - \alpha)B$$

- Taking partial derivatives:

$$\nabla I = (F - B)\nabla \alpha + \alpha \nabla F + (1 - \alpha)\nabla B$$

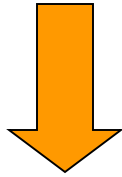
- Case I: foreground and background have much smaller gradient values

$$\nabla I \approx (F - B)\nabla \alpha$$

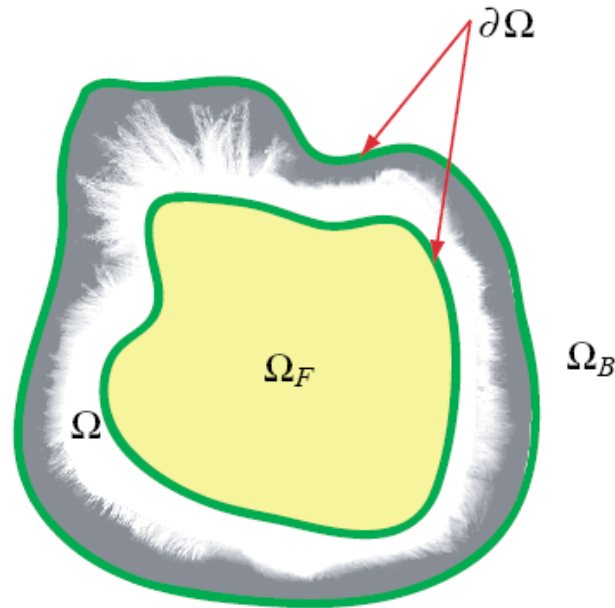
$$\nabla \alpha \approx \frac{\nabla I}{F - B}$$

Poisson Matting

$$\nabla \alpha \approx \frac{\nabla I}{F - B}$$

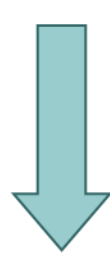


$$\Delta \alpha = \operatorname{div}\left(\frac{\nabla I}{F - B}\right) \quad \text{s.t. } \alpha|_{\partial\Omega} = \begin{cases} 1 & \mathbf{x} \in \Omega_F \\ 0 & \mathbf{x} \in \Omega_B \end{cases}$$



Poisson Matting

$$\alpha^* = \arg \min_f \iint_{\Omega} \left\| \nabla \alpha - \frac{\nabla I}{F - B} \right\|^2 \quad \text{s.t. } \alpha^*|_{\partial\Omega} = \alpha|_{\partial\Omega}$$



J

Euler Equation: $J_f - \frac{\partial}{\partial x} J_{f_x} - \frac{\partial}{\partial y} J_{f_y} = 0$

$$\Delta \alpha = \operatorname{div} \left(\frac{\nabla I}{F - B} \right) \quad \text{s.t. } \alpha|_{\partial\Omega} = \alpha|_{\partial\Omega}$$

$\frac{\nabla I}{F - B}$ is a **guidance** field, an approximation of matte gradient.

Image Co-Segmentation

- Task specification: given an image pair, detect the common foreground in the set of images despite the differences in the background



Image Co-Segmentation

- The first work on image co-segmentation (?)



Input Image pair



GrabCut on individual images



Cosegmentation, no spatial consistency



Cosegmentation

Rother et al., Cosegmentation of Image Pairs by Histogram Matching - Incorporating a Global Constraint into MRFs, CVPR 2006

Image Co-Segmentation

- Application: robust image distance

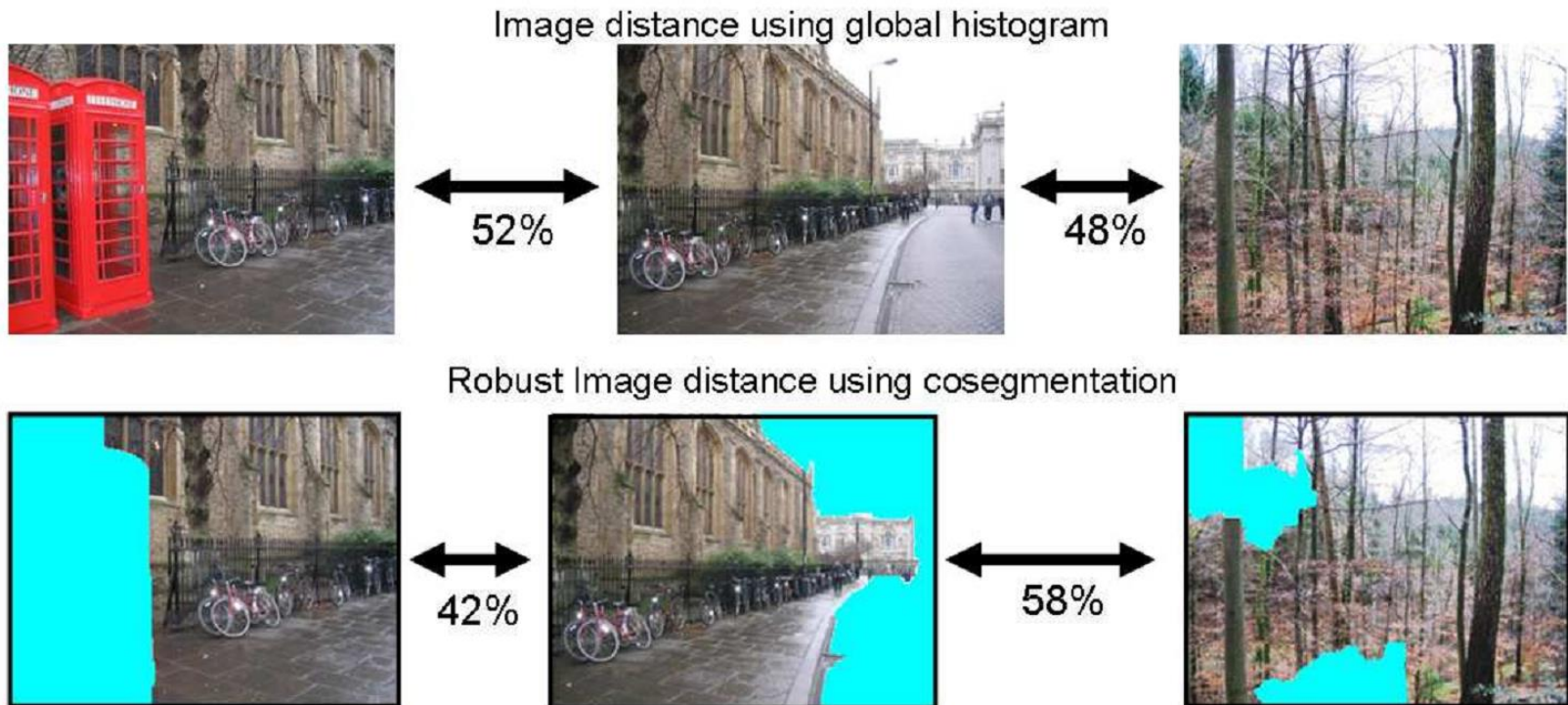


Image Co-Segmentation

- Application: robust image distance

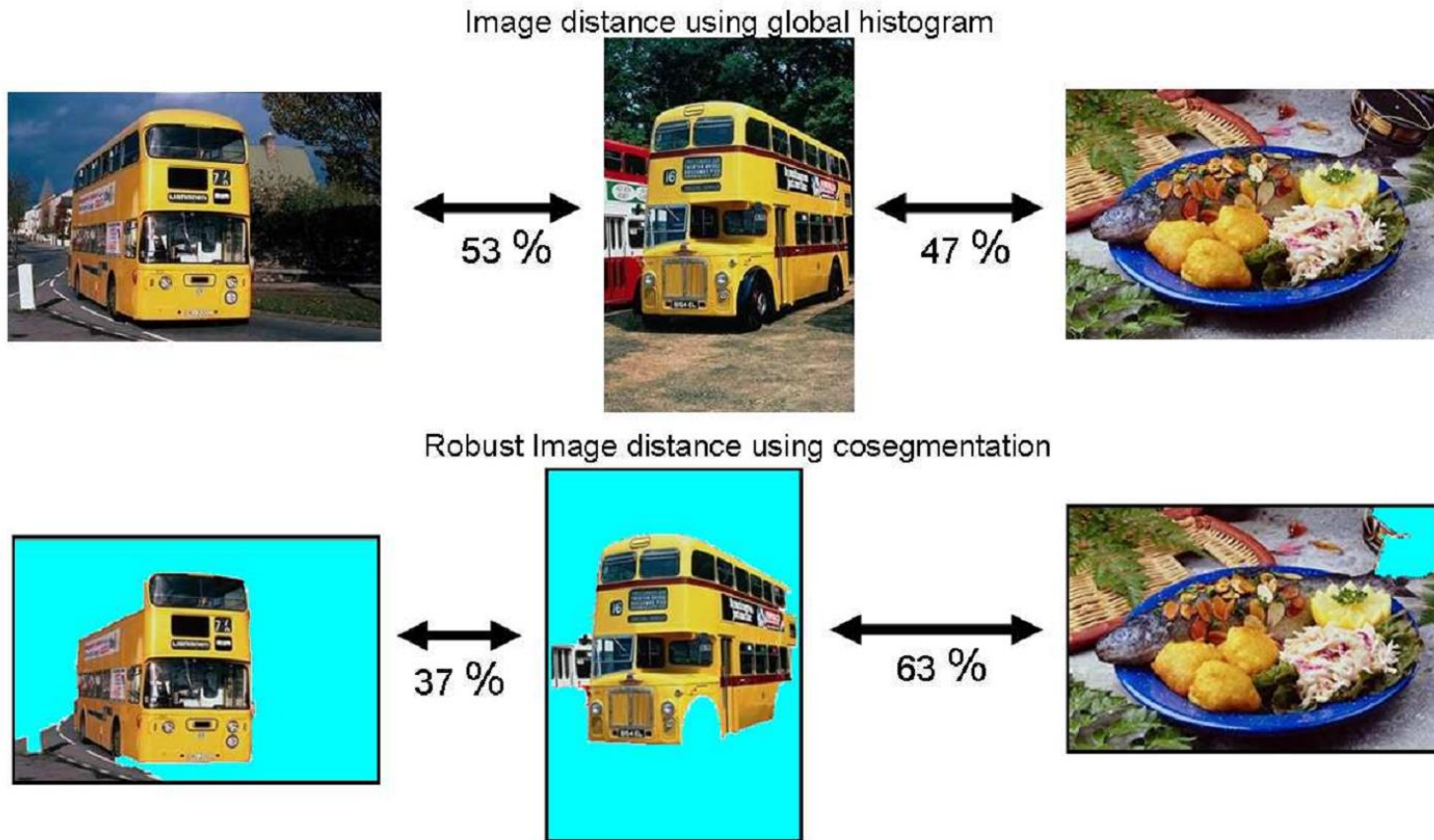


Image Inpainting / Image Completion

- What motivates image inpainting / completion techniques?



Some Examples



Some Examples



Photoshop CS5 “Content-Aware Fill”



before



after

YouTube demos: <http://www.youtube.com/watch?v=PTvxlFBW96k>

The Difficulties

- Under-constrained problem
- Requires inference about regions, boundaries, textures
- Data-driven but needs scene semantics
- Result should “look real” without seams or perceptual artifacts

Intuitive Solution?

- Identify the regions you want to fill/remove
- Continue any lines arriving at those regions
- Fill in the regions with texture/color from the surrounding areas

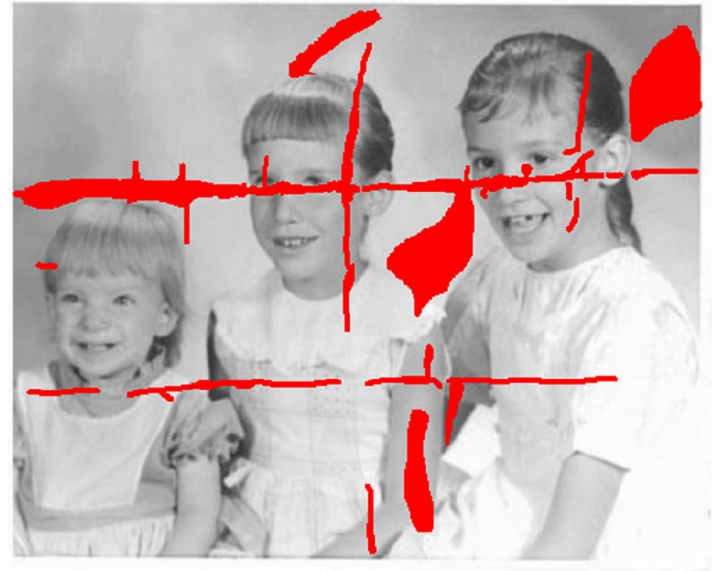


Image Inpainting

Marcelo Bertalmio and Guillermo Sapiro*

Electrical and Computer Engineering, University of Minnesota

Vicent Caselles and Coloma Ballester

Escola Superior Politecnica, Universitat Pompeu Fabra





Structure vs. Texture



Object Removal by Exemplar-Based Inpainting

A. Criminisi, P. Pérez

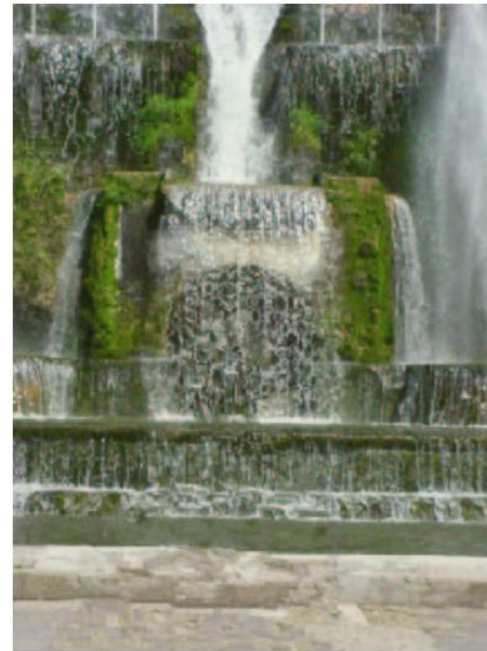
Microsoft Research Ltd., Cambridge, UK

`antcrim@microsoft.com, pperez@microsoft.com`

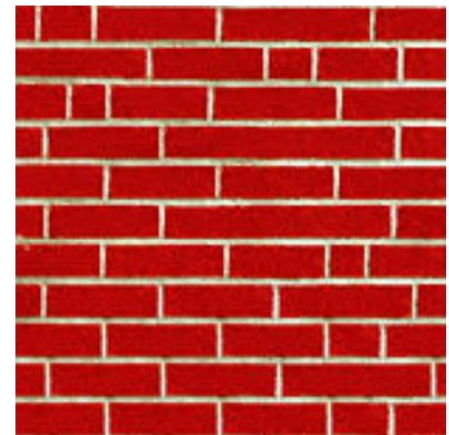
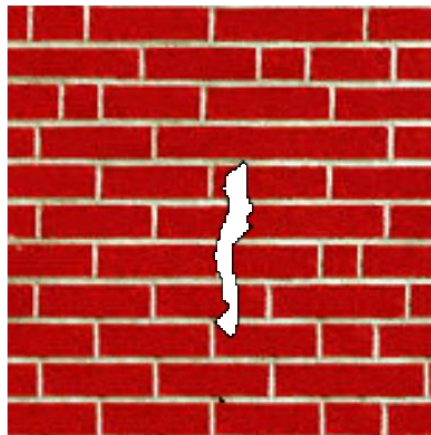
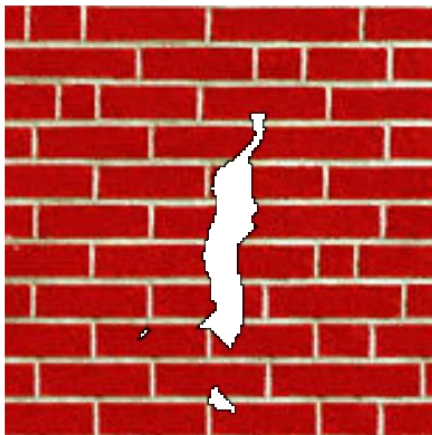
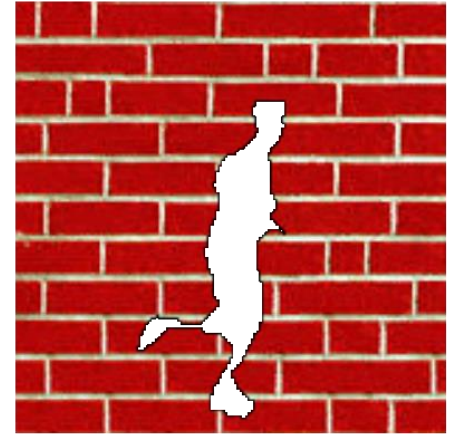
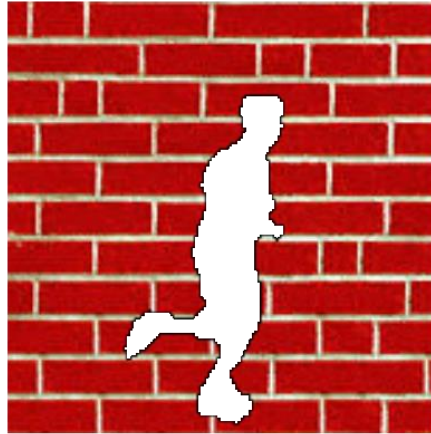
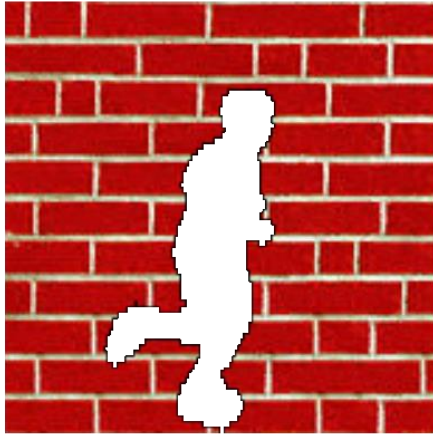
K. Toyama

Microsoft Corporation, Redmond, WA, USA

`kentoy@microsoft.com`

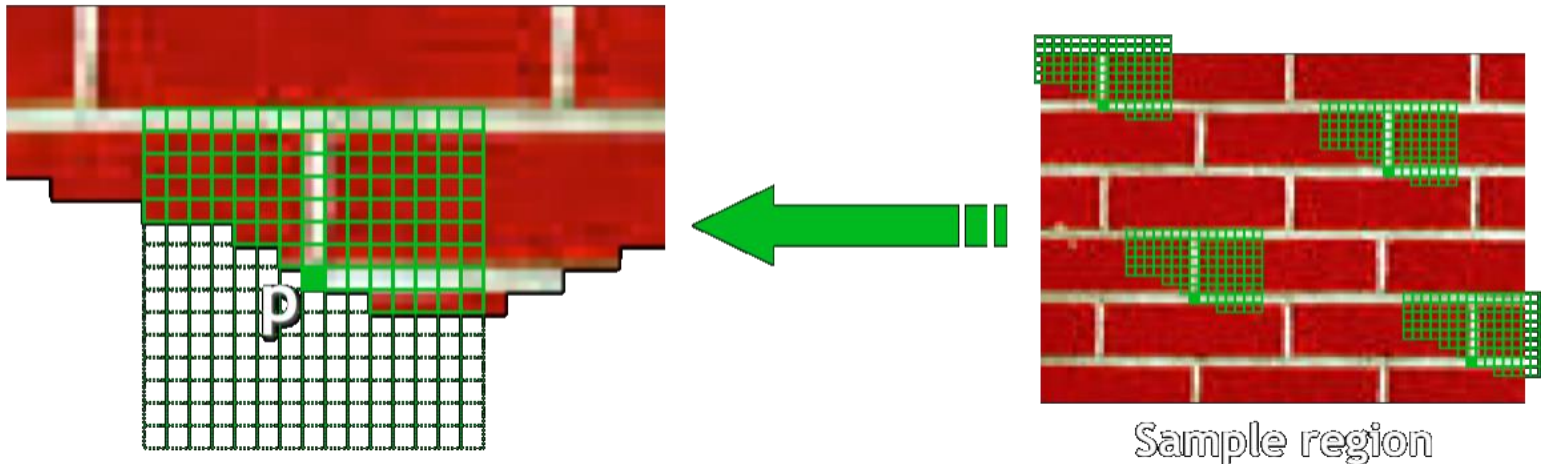


“Onion Skin” Filling Order



Hole Filling by Examples

- Cheap and easy, but the filling order is crucial in preserving structure



Filling in “Onion Skin” Order



May lose linear structure

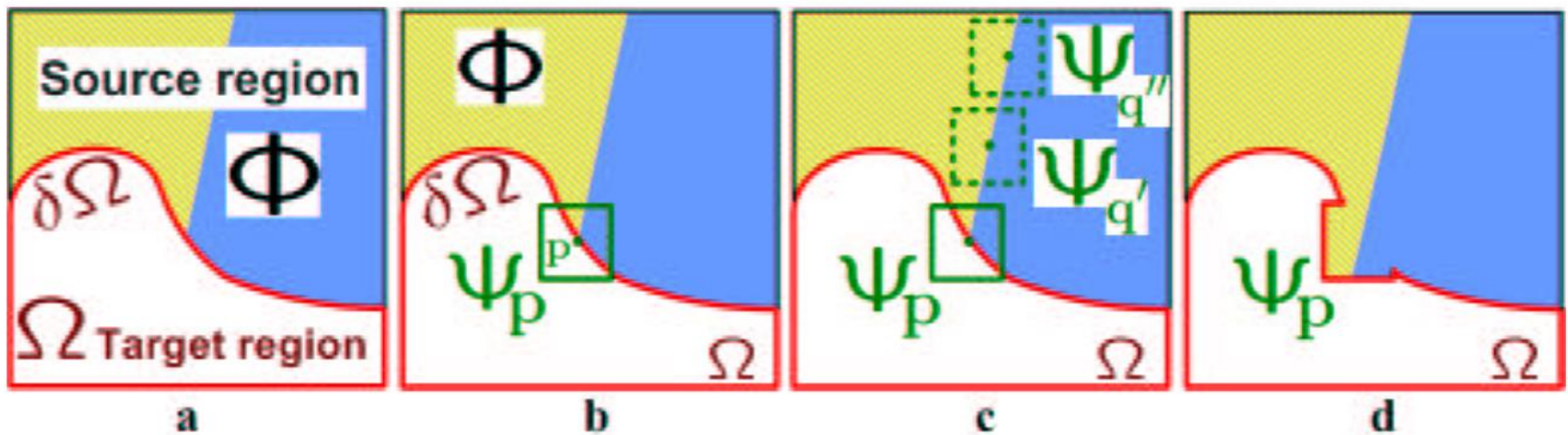
Filling Order Is Crucial



choose pixels that are continuations of lines/curves/edges so that linear structures are propagated

Algorithm Pipeline

- Extract the initial fill-front, $\delta\Omega$, and compute priority values, $P(\mathbf{p})$, for all pixels $\mathbf{p}$ in $\delta\Omega$
- Repeat until no more unfilled pixels remain:
 1. Find target pixel $\mathbf{p}$ and patch $\Psi_{\mathbf{p}}$ with highest “priority,” $P(\mathbf{p})$
 2. Find source patches, $\Psi_{\mathbf{q}}$, $\Psi_{\mathbf{q}'}$, that most closely match the target patch
 3. Paste most similar source patch into target location
 4. Update fill front and priority values



Determining Filling Priority

- Assign each pixel $\mathbf{p}$ centered on a patch $\Psi_{\mathbf{p}}$ a **priority value** $P(\mathbf{p})$ that is a product of a confidence term, $C(\mathbf{p})$, and data term, $D(\mathbf{p})$:

$$P(\mathbf{p}) = C(\mathbf{p})D(\mathbf{p})$$

Measures fraction of P 's neighbors already filled

Confidence term:

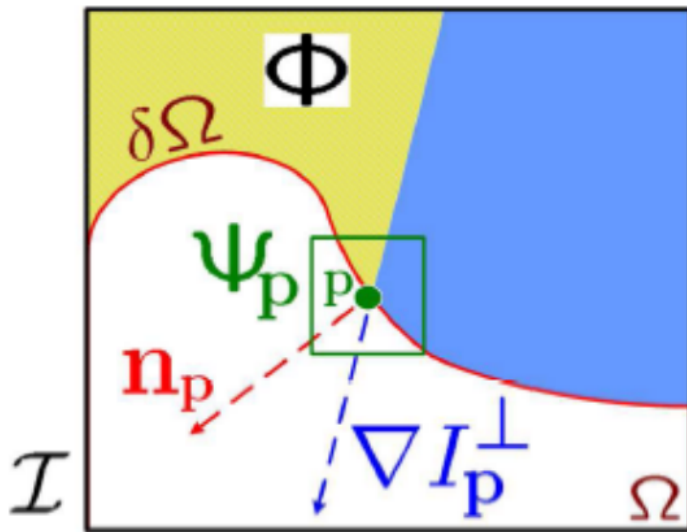
$$C(\mathbf{p}) = \frac{\sum_{\mathbf{q} \in \Psi_{\mathbf{p}} \cap \bar{\Omega}} C(\mathbf{q})}{|\Psi_{\mathbf{p}}|}$$

$$\bar{\Omega} = I - \Omega$$

Data term:

$$D(\mathbf{p}) = \frac{|\nabla I_{\mathbf{p}}^{\perp} \cdot \mathbf{n}_{\mathbf{p}}|}{\alpha}$$

Measures strength and orientation of edge at P



C measures fraction of patch already filled

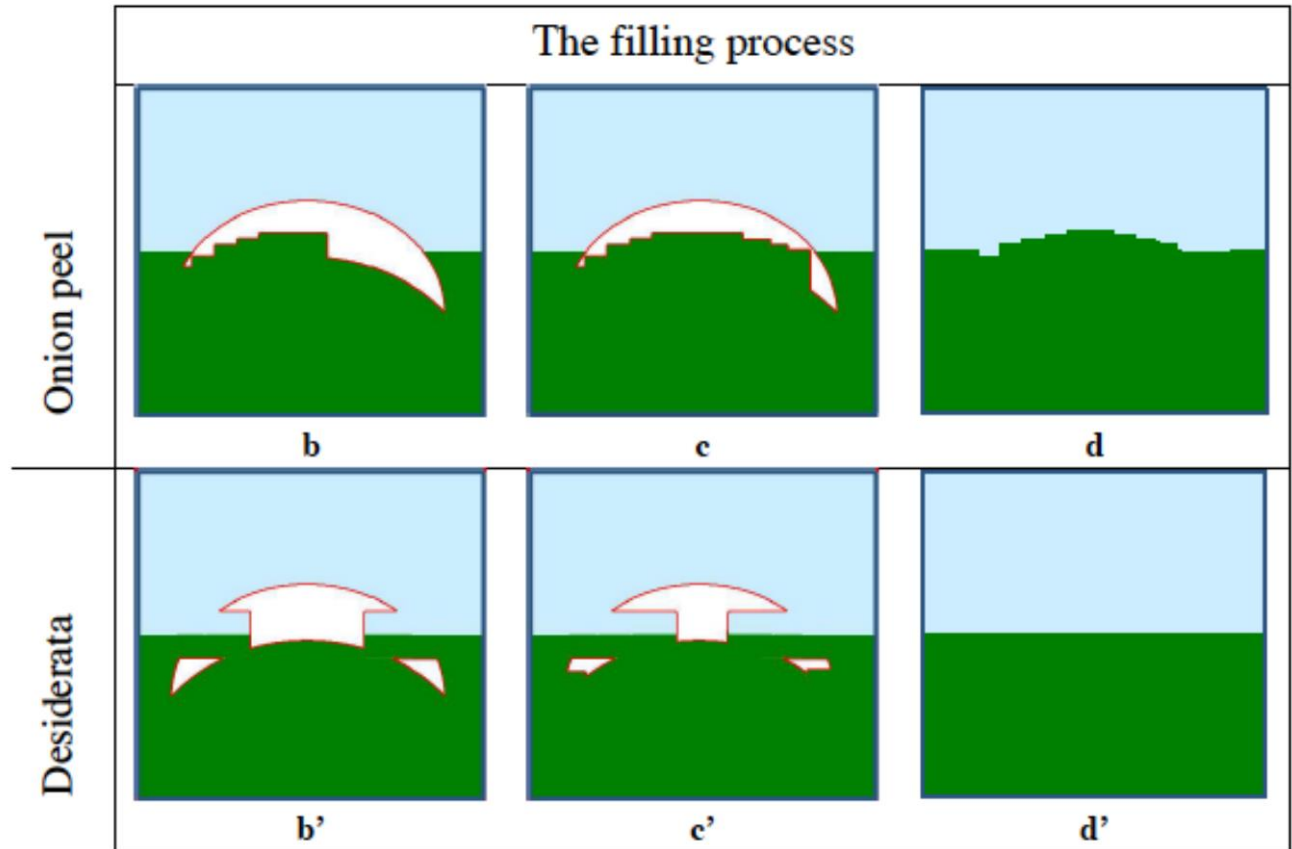
D measures the degree to which an edge ends at $\mathbf{p}$

α is a constant

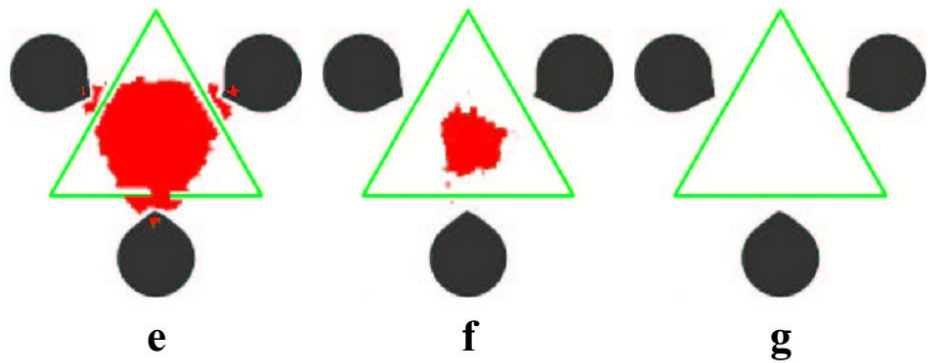
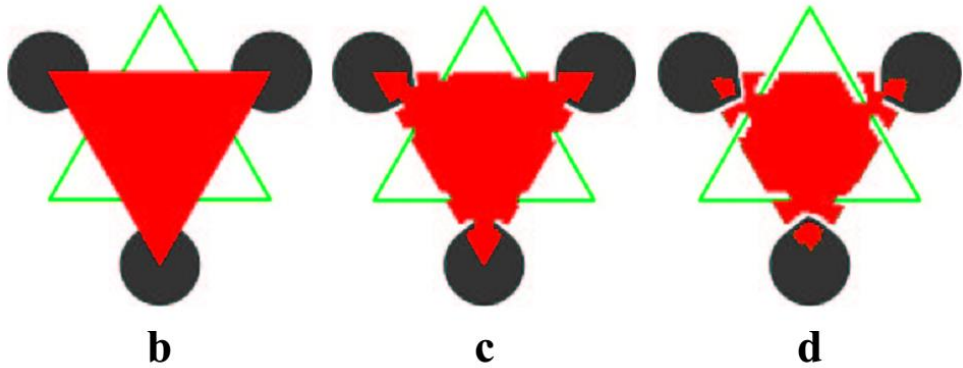
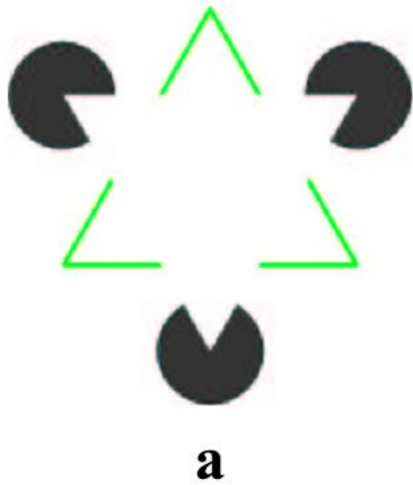
Onion Skin vs. Structure-Keeping



a



“Kanizsa Triangle”



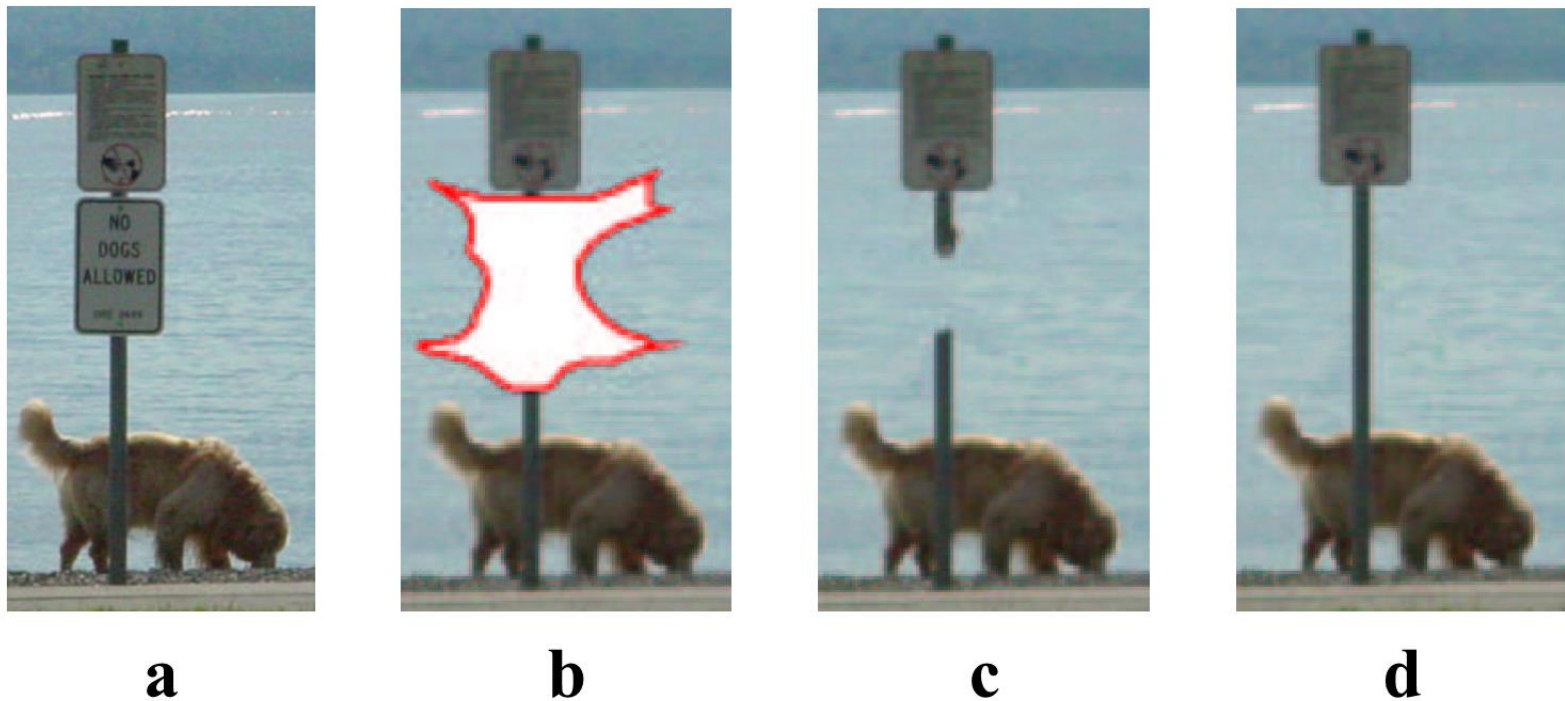
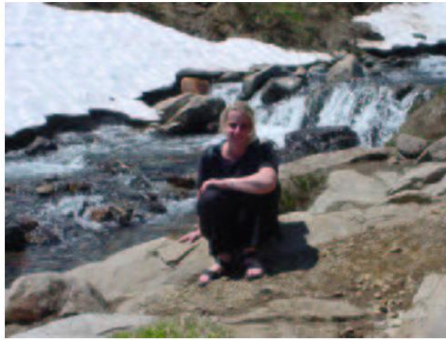


Figure 7: Onion peel vs. structure-guided filling. (a) Original image. (b) The target region has been selected and marked in red. (c) Results of filling by concentric layers. (d) Results of filling with our algorithm. Thanks to the *data term* in (1) the pole is reconstructed correctly.

Do We Create Something New?



a



b



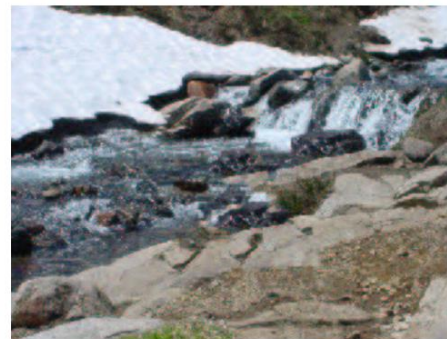
c



d

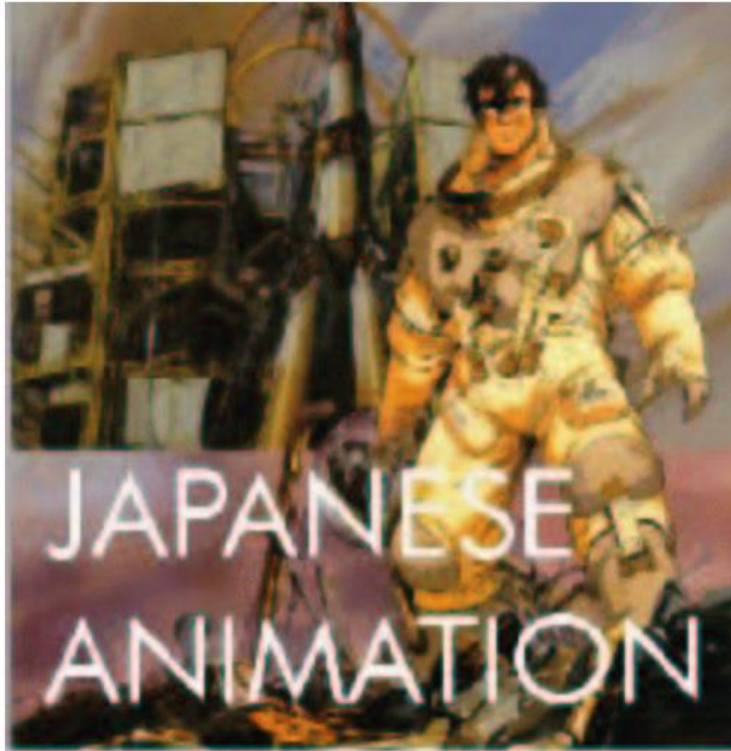


e

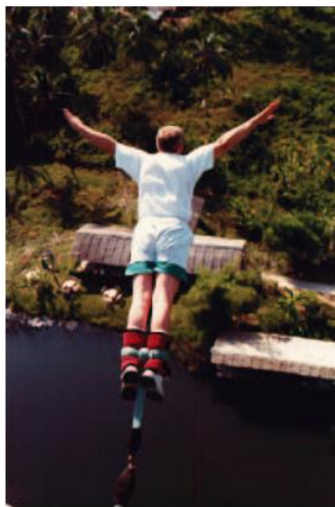


f

More Results



More Results



a



b



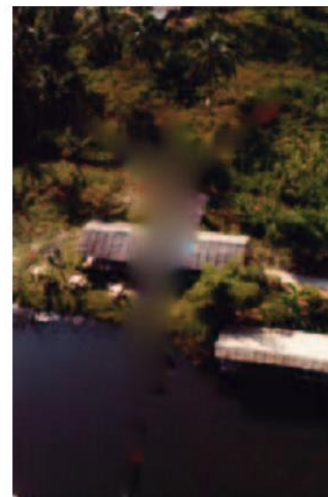
c



d



e



f

Image Completion with Structure Propagation

Jian Sun¹ Lu Yuan^{2*} Jiaya Jia^{3†} Heung-Yeung Shum¹

¹Microsoft Research Asia ²Tsinghua University ³Chinese University of Hong Kong



(a)



(b)



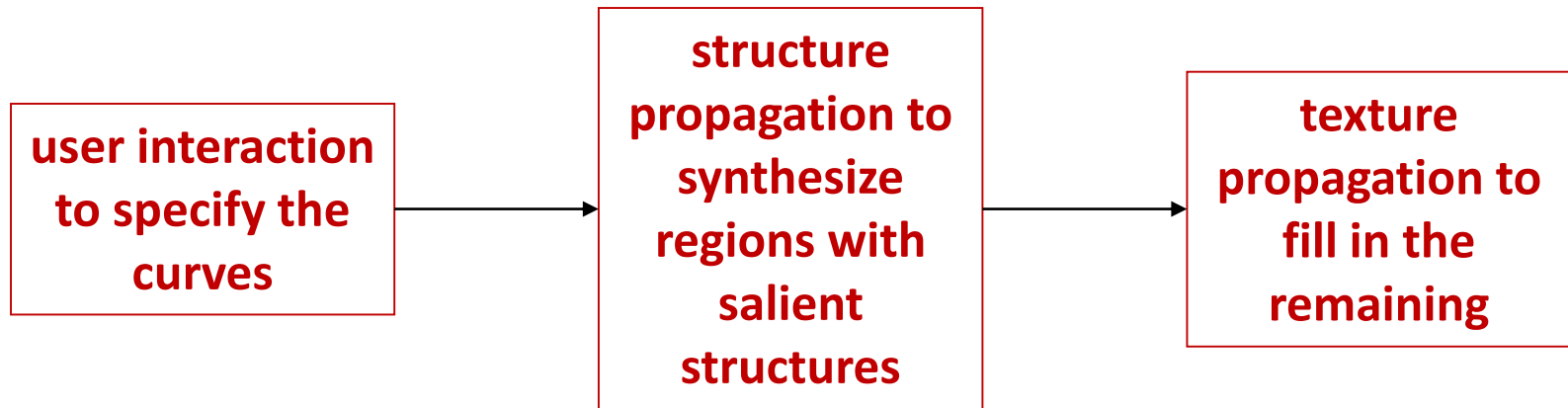
(c)



(d)

Major Observations

- For natural images, the most salient missing structures can often be approximated by a few well-defined curves.
- There exists a synthesis ordering for image completion: the regions with salient structures should be completed before filling in other regions.



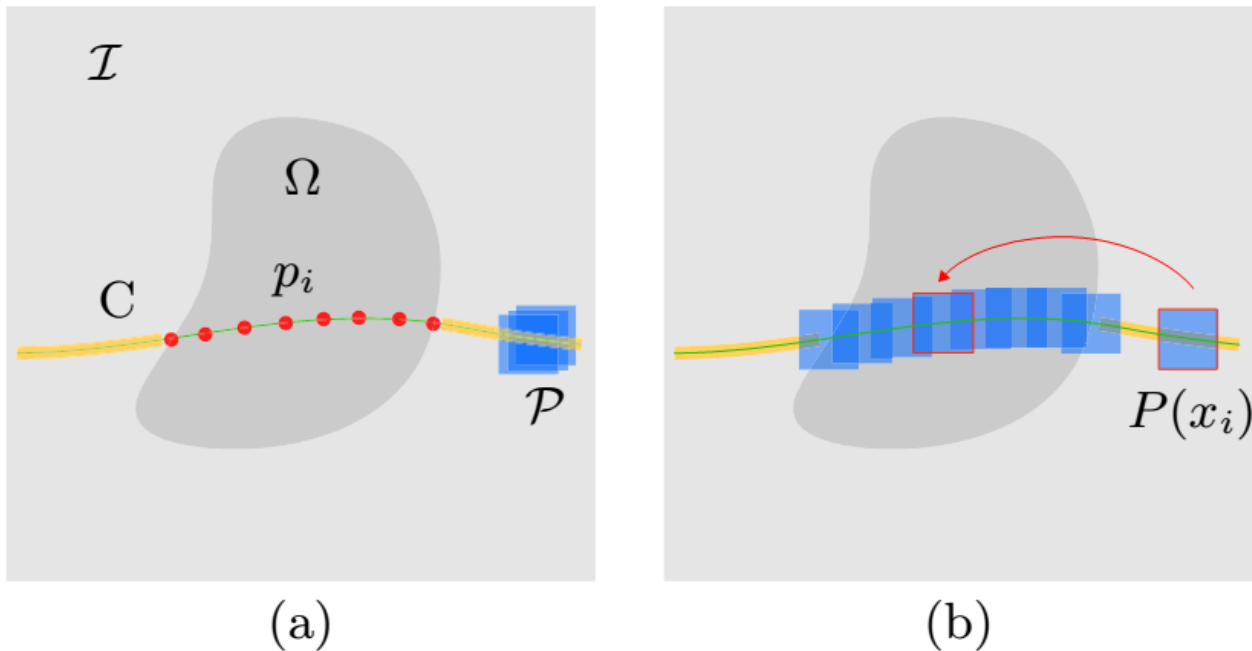


Figure 2: Structure propagation - 1D chain. (a) $\mathcal{I}$ is the input image region, Ω is the unknown region and C is a user-specified curve. Structure propagation synthesizes missing image patches on a set of anchor points $\{p_i\}_{i=1}^L$ using the sample set $\mathcal{P}$. (b) $P(x_i)$ is a candidate patch in $\mathcal{P}$ which is chosen for the anchor point p_i .

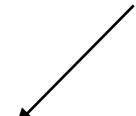
Formulating as Graph Labeling

- Sample anchors / source patches along the user-specified curves
- Assign label (source patch) to each anchor

We define the following energy on $\mathcal{G}$

$$E(X) = \sum_{i \in \mathcal{V}} E_1(x_i) + \sum_{(i,j) \in \mathcal{E}} E_2(x_i, x_j), \quad (1)$$

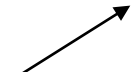
Coherence term



where

$$E_1(x_i) = k_s \cdot E_S(x_i) + k_i \cdot E_I(x_i). \quad (2)$$

Structure term



Completion term



Energy Terms

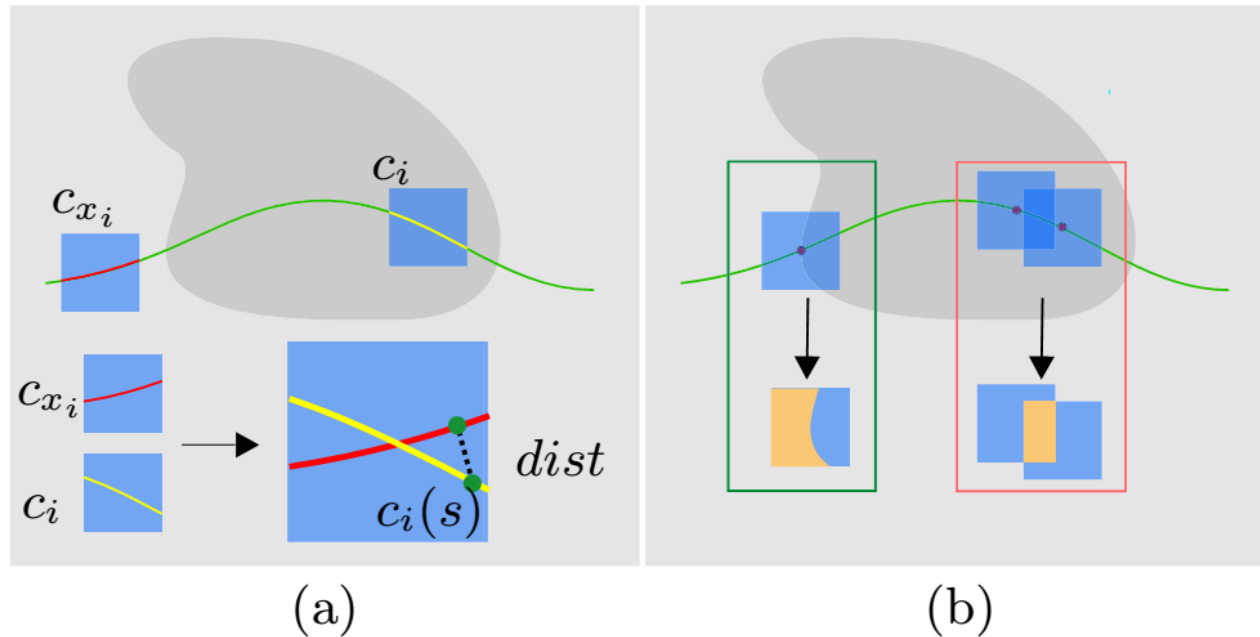


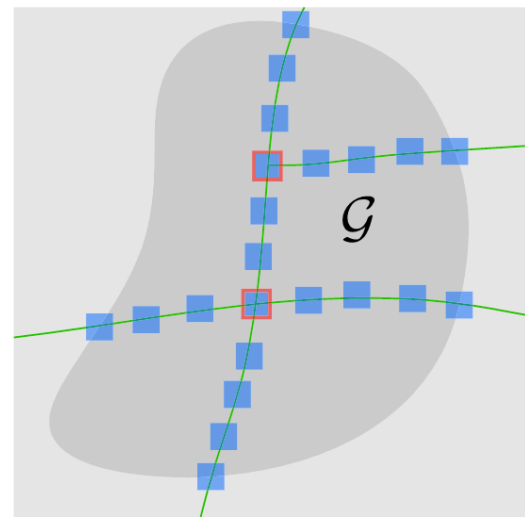
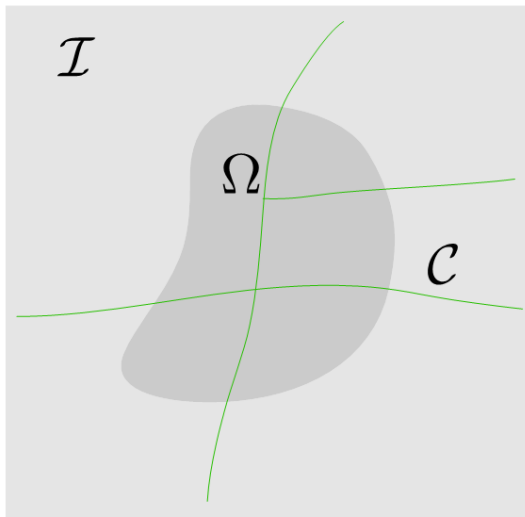
Figure 3: Energy terms for structure propagation. (a) Curve segments c_{x_i} (red) in the source patch, and curve segments c_i (yellow) in the target rectangle. $E_S(x_i)$ measures the structure similarity between c_{x_i} and c_i . $dist$ is the shortest distance (black dotted line) from point $c_i(s)$ on segment c_i to segment c_{x_i} . (b) The green box shows the cost $E_I(x_i)$ on the boundary of the unknown region. The red box shows the cost $E_2(x_i, x_j)$ for neighboring patches.

Dynamic Programming (DP) Solver

- For 1-D chain, DP works

$$M_i(x_i) = E_1(x_i) + \min_{x_{i-1}} \{E_2(x_{i-1}, x_i) + M_{i-1}(x_{i-1})\}$$

- How about general graphs?



The BP Algorithm

Algorithm 1 Completion using Belief Propagation.

- 1: Initialize all messages $M_{ij}^0 = \mathbf{0}$ between any two adjacent nodes i and j in graph $\mathcal{G}$.
- 2: Update all messages M_{ij}^t iteratively from $t = 1$ to T :

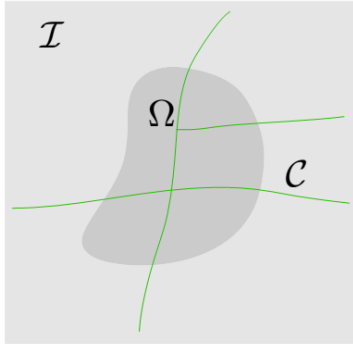
$$M_{ij}^t = \min_{x_i} \{E_1(x_i) + E_2(x_i, x_j) + \sum_{k \neq j, k \in \mathcal{N}(i)} M_{ki}^{t-1}\} \quad (5)$$

where $\mathcal{N}(i)$ are all adjacent nodes of i .

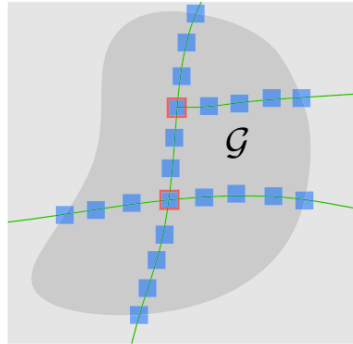
- 3: Compute optimal label x_i^* for each node i :

$$x_i^* = \arg \min_{x_i} \{E_1(x_i) + \sum_{k \in \mathcal{N}(i)} M_{ki}^T\} \quad (6)$$

An Example

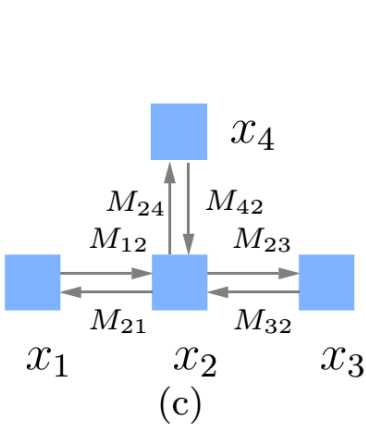


(a)

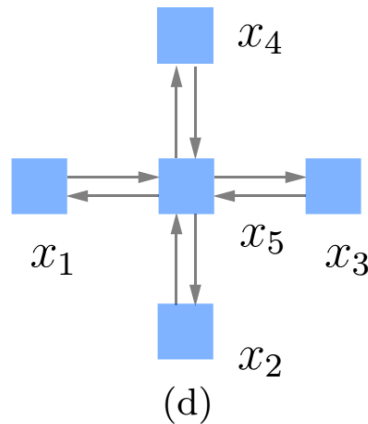


(b)

$$\begin{aligned} x_1^* &= \arg \min_{x_1} \{E_1(x_1) + M_{21}\} \\ x_2^* &= \arg \min_{x_2} \{E_1(x_2) + M_{12} + M_{32} + M_{42}\} \\ x_3^* &= \arg \min_{x_3} \{E_1(x_3) + M_{23}\} \\ x_4^* &= \arg \min_{x_4} \{E_1(x_4) + M_{24}\} \end{aligned}$$



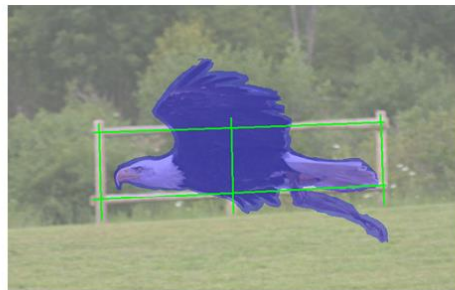
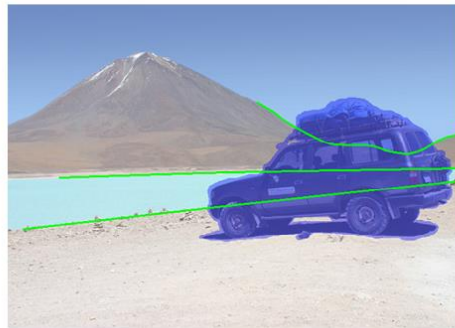
(c)



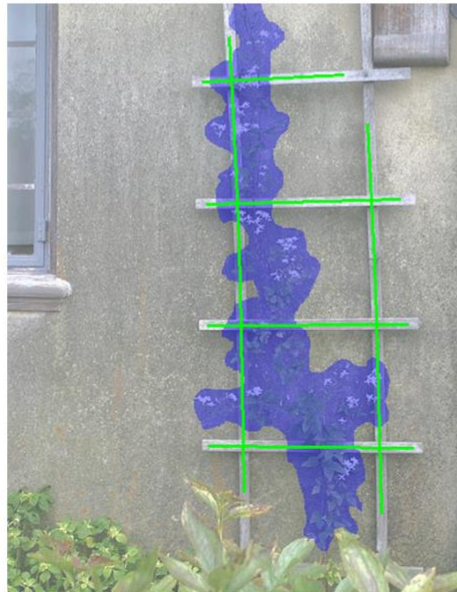
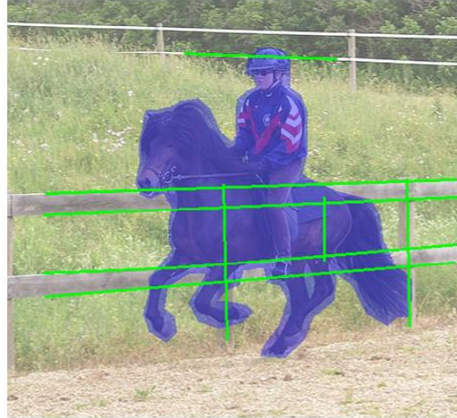
(d)

$$\begin{aligned} M_{12} &= \min_{x_1} \{E_1(x_1) + E_2(x_1, x_2)\} \\ M_{32} &= \min_{x_3} \{E_1(x_3) + E_2(x_2, x_3)\} \\ M_{42} &= \min_{x_4} \{E_1(x_4) + E_2(x_2, x_4)\} \\ M_{21} &= \min_{x_2} \{E_1(x_2) + E_2(x_1, x_2) + M_{32} + M_{42}\} \\ M_{23} &= \min_{x_2} \{E_1(x_2) + E_2(x_2, x_3) + M_{12} + M_{42}\} \\ M_{24} &= \min_{x_2} \{E_1(x_2) + E_2(x_2, x_4) + M_{12} + M_{32}\} \end{aligned}$$

Sampled Results



Sampled Results



Layer-Sensitive Completion



Questions?